

PROJET DE FIN D'ÉTUDES

DIPLÔME NATIONAL D'INGÉNIEUR

SPÉCIALITÉ : Informatique (TWIN)

**Conception et développement
d'une plateforme décisionnelle
ressource humaine (RH) &
Crédit pour la Banque
Extérieure d'Algérie**

Réalisé par : Mohamed Yassine Mighri
Encadrante académique : Olfa Layouni
Encadrant professionnel : Mouadh Zidi

Année universitaire :
2025/2026



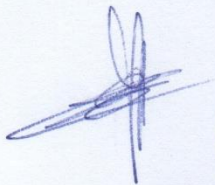
Je valide le dépôt du rapport PFE relatif à l'étudiant nommé ci-dessous / I validate the submission of the student's report:

- Nom & Prénom /Name & Surname : Mighri Mohamed Yassine

Encadrant Entreprise/ Business site Supervisor

- Nom & Prénom /Name & Surname : Zidi Mouadh

Cachet & Signature / Stamp & Signature



Encadrant Académique/Academic Supervisor

- Nom & Prénom /Name & Surname : Layouni Olfa

Signature / Signature

Olfa Layouni

Ce formulaire doit être rempli, signé et **scanné**/This form must be completed, signed and **scanned**.

Ce formulaire doit être introduit après la page de garde/ This form must be inserted after the cover page.

Dédicaces

À mes parents, Mohamed Mongi et Nejla

À vous qui avez toujours cru en moi, qui m'avez soutenu dans chaque étape de mon parcours et qui avez fait tant de sacrifices pour me permettre d'avancer. Je vous dédie ce travail avec tout mon amour, ma reconnaissance et mon profond respect. Votre soutien a été ma plus grande force et votre confiance, ma motivation pour aller toujours plus loin.

À mes sœurs, Yosr et Nour

À celles avec qui j'ai partagé tant de souvenirs, de rires et de moments précieux. Merci pour votre amour, votre patience et votre soutien tout au long de mon parcours. Je suis reconnaissant de vous avoir à mes côtés et fier de pouvoir partager cette réussite avec vous.

À tous mes enseignants de l'ESPRIT,

Pour les connaissances que vous m'avez transmises, votre accompagnement et votre contribution à ma formation. Merci de m'avoir permis de développer non seulement mes compétences, mais aussi ma passion pour le métier d'ingénieur.

À mes amis,

Pour tous les moments partagés, les longues heures de travail, les difficultés surmontées et les réussites célébrées ensemble. Merci pour votre présence et votre soutien tout au long de ce parcours.

À toutes les personnes qui, de près ou de loin, ont contribué à la réalisation de ce projet,

Je vous adresse mes sincères remerciements et vous dédie ce travail avec toute ma gratitude.

À vous tous, je dédie ce travail.

Remerciements

Je tiens à exprimer ma profonde gratitude à l'ensemble des personnes qui ont contribué, de près ou de loin, à la réussite de ce stage de fin d'études ainsi qu'à la rédaction de ce rapport.

Je remercie tout d'abord **ARABSOFT** de m'avoir accueilli en son sein et de m'avoir offert un cadre de travail stimulant, professionnel et enrichissant.

Je souhaite adresser mes plus sincères remerciements à **Monsieur Mouadh Zidi**, mon encadrant au sein de l'entreprise, pour son accompagnement, sa disponibilité et ses précieux conseils tout au long de ce stage. Son expertise, son soutien et sa confiance ont été essentiels à la réussite de ce projet.

J'adresse également mes sincères remerciements à **Madame Olfa Laayouni**, mon encadrante académique, pour ses orientations, son suivi rigoureux et ses remarques constructives tout au long de ce projet. Ses conseils avisés ont grandement contribué à l'amélioration de mon travail.

Je remercie également les membres du jury pour le temps qu'ils consacrent à l'évaluation de ce travail, ainsi que pour l'intérêt qu'ils portent à ce projet.

Enfin, j'adresse mes remerciements les plus chaleureux à ma famille et à mes proches et à mes amis pour leur soutien indéfectible, leurs encouragements constants et leur patience tout au long de mon parcours académique.

Table des matières

Introduction générale

Chapitre 1 : Contexte général du projet

1.1 Présentation de l'organisme d'accueil

1.2 Contexte et problématique

1.3 Objectifs du stage

1.4 Périmètre fonctionnel du projet

1.4.1 Domaine des RH

1.4.2 Domaine du Crédit

1.4.3 Gestion des absences

1.5 Méthodologie de travail adoptée

1.6 Conclusion

Chapitre 2 : État de l'art et concepts théoriques

2.1 Introduction à la Business Intelligence (BI)

2.2 Entrepôts de données (Data Warehouse) et modélisation
décisionnelle

2.2.1 Schéma en étoile

2.2.2 Tables de faits et dimensions

2.2.3 Organisation du schéma décisionnel

2.2.4 Avantages du schéma en étoile

2.3 Processus ETL (Extract – Transform – Load)

2.3.1 Extraction

2.3.2 Transformation

2.3.3 Chargement

2.4 Architectures des applications web modernes

- 2.4.1 API REST
- 2.4.2 Applications SPA (Single Page Application)
- 2.5 Sécurité et gestion des identités
 - 2.5.1 OAuth 2.0
 - 2.5.2 Keycloak
- 2.6 Conclusion

Chapitre 3 : Analyse et spécification des besoins

- 3.1 Étude de l'existant
- 3.2 Identification des acteurs et de leurs rôles
- 3.3 Spécification des besoins fonctionnels
- 3.4 Spécification des besoins non fonctionnels
- 3.5 Diagrammes des cas d'utilisation
- 3.6 Conclusion

Chapitre 4 : Conception de la plateforme décisionnelle

- 4.1 Analyse des sources de données
- 4.2 Conception du Data Warehouse
- 4.3 Conception du processus ETL
- 4.4 Conception des indicateurs de performance (KPIs RH et Crédit)
- 4.5 Conclusion

Chapitre 5 : Conception de l'architecture applicative

- 5.1 Architecture globale de la solution
- 5.2 Conception de l'API Backend (Spring Boot)

5.3 Modélisation des données (Diagramme de classes)

5.4 Conception de l'interface utilisateur (Angular)

5.5 Architecture de sécurité et gestion des rôles (Keycloak)

5.6 Conclusion

Chapitre 6 : Réalisation et implémentation

6.1 Environnement de développement et outils utilisés

6.2 Mise en œuvre du Data Warehouse et du processus ETL

6.3 Développement de l'API REST (Spring Boot, Spring Data JPA)

6.4 Développement du tableau de bord (Angular, ApexCharts/Chart.js)

6.5 Mise en place de l'authentification et de la gestion des rôles (Keycloak)

6.6 Conteneurisation et déploiement avec Docker et Docker Compose

6.7 Présentation des interfaces réalisées

6.8 Conclusion

Chapitre 7 : Tests et validation

7.1 Stratégie de tests

7.2 Tests unitaires et fonctionnels

7.3 Tests de performance

7.4 Validation des indicateurs décisionnels (KPIs)

7.5 Conclusion

Conclusion générale et perspectives

Liste des figures

Figure 1: Logo ARABSOFT

Figure 2: Data Warehouse Architecture

Figure 3 : Diagramme des cas d'utilisation de la plateforme décisionnelle

Figure 4 : Architecture fonctionnelle

Figure 5 : Schémas en étoile du Data Warehouse RH et Crédit

Figure 6 : Processus ETL détaillé

Figure 7 : Architecture globale de la plateforme décisionnelle RH et Crédit

Figure 8: Architecture de l'API Spring Boot

Figure 9 : Diagramme de classes – Modèle de données du Data Warehouse RH et Crédit

Figure 10 : Interface d'authentification

Figure 11 : Tableau de bord principal

Figure 12 : Dashboard

Absences **Figure13 :** Dashboard Effectifs

Figure14 : Dashboard Prêt

Figure 15 : Dashboard Salaires

Liste d'abréviation

Abréviation	Signification
API	Application Programming Interface
BEA	Banque Extérieure d'Algérie
BI	Business Intelligence
DTO	Data Transfer Object
ETL	Extract, Transform, Load
HTTP	HyperText Transfer Protocol
HTTPS	HyperText Transfer Protocol Secure
IAM	Identity and Access Management
JPA	Java Persistence API
JSON	JavaScript Object Notation
JWT	JSON Web Token
REST	Representational State Transfer
RH	Ressources Humaines
SCD	Slowly Changing Dimension
SPA	Single Page Application
SSO	Single Sign-On
URI	Uniform Resource Identifier

Introduction générale

Les banques produisent chaque jour de gros volumes de données grâce à leurs services, notamment les ressources humaines et le crédit. Ces données sont souvent réparties dans plusieurs systèmes, ce qui complique leur analyse et leur utilisation pour prendre des décisions. Dans ce contexte, notre projet de fin d'études, réalisé à la Banque Extérieure d'Algérie, vise à créer une plateforme décisionnelle. Cette plateforme doit centraliser, analyser et visualiser les données RH et crédit à l'aide de tableaux de bord interactifs destinés aux responsables RH, finance et direction.

Pour répondre à ce besoin, nous combinons trois parties complémentaires : un Data Warehouse alimenté par un processus ETL, une API backend qui donne accès aux données, et une application web de type dashboard avec authentification, gestion des rôles et filtres dynamiques.

Le rapport se compose de sept chapitres. Le premier décrit le contexte général et les objectifs du projet. Le deuxième expose les concepts théoriques de la Business Intelligence. Le troisième analyse les besoins. Les quatrième et cinquième chapitre présentent respectivement la conception de la solution décisionnelle et l'architecture applicative. Le sixième chapitre détaille la réalisation du projet, et le septième rend compte des tests effectués. Le rapport se conclut par une synthèse générale et des perspectives.

Chapitre 1 : Contexte général du projet

1.1 Présentation de l'organisme d'accueil

ARABSOFT (Société Arabe de Logiciels) est une entreprise tunisienne spécialisée dans le développement de solutions logicielles et de systèmes d'information. Fondée en 1988, elle accompagne les organismes publics et les entreprises dans leur transformation numérique en proposant des solutions innovantes dans divers domaines, notamment la gestion des RH, les finances, la santé et l'administration électronique. Grâce à son expertise et à son engagement envers la qualité, ARABSOFT est aujourd'hui un acteur majeur du secteur des technologies de l'information en Tunisie et dans plusieurs pays arabes.



Figure 1 : Logo ARABSOFT

1.2 Contexte et problématique

Dans un contexte où les organisations génèrent chaque jour un volume important de données, leur valorisation constitue un enjeu majeur pour améliorer la prise de décision. Les établissements financiers sont particulièrement concernés. Ils doivent disposer d'outils performants pour exploiter efficacement les données liées aux ressources humaines (RH) et aux activités de crédit, et ainsi renforcer leur pilotage stratégique.

La Banque Extérieure d'Algérie (BEA) dispose de nombreuses données issues de différents systèmes d'information. Elles concernent notamment le personnel, les charges salariales, les crédits et les absences. Toutefois, ces informations sont réparties dans plusieurs tables de la base de données opérationnelle. Cette organisation complique leur exploitation analytique, qui devient alors plus longue et moins adaptée aux besoins des décideurs.

Pour répondre à cette problématique, la mise en place d'une plateforme décisionnelle apparaît nécessaire. Celle-ci doit centraliser et transformer les données afin de produire des informations fiables et exploitables. Elle doit également permettre leur analyse à travers des indicateurs pertinents et des tableaux de bord interactifs. Les responsables RH et financiers pourront ainsi suivre les principaux indicateurs de performance et effectuer des analyses selon différents critères, tels que la période, le service ou le grade. La prise de décision sera ainsi facilitée et mieux appuyée par les données.

C'est dans ce contexte que s'inscrit ce projet de fin d'études. Il a pour objectif de concevoir et de développer une plateforme décisionnelle destinée aux responsables RH et financiers de la Banque Extérieure d'Algérie. La solution proposée intègre un Data Warehouse, des processus ETL, une API de services ainsi qu'une application web sécurisée dédiée à la visualisation et à l'analyse des données.

1.3 Objectifs du stage

Ce stage a pour objectif principal de concevoir et de développer une plateforme décisionnelle dédiée à l'analyse des données relatives aux ressources humaines (RH) et aux activités de crédit de la Banque Extérieure d'Algérie (BEA). La plateforme a pour vocation de centraliser les données issues des systèmes opérationnels, de les intégrer dans un Data Warehouse et de mettre à disposition des tableaux de bord interactifs. Ces outils permettent aux responsables de suivre les principaux indicateurs clés de performance et de disposer d'informations utiles à la prise de décision.

Pour atteindre cet objectif, plusieurs travaux ont été réalisés au cours du stage :

- Concevoir un Data Warehouse reposant sur une modélisation décisionnelle en schéma en étoile.
- Développer des processus ETL dédiés à l'extraction, à la transformation et au chargement des données.
- Concevoir et développer une API REST sécurisée pour permettre l'accès aux données analytiques.
- Développer une application web intégrant des tableaux de bord interactifs pour visualiser les indicateurs liés aux RH et au Crédit.
- Mettre en place un mécanisme d'authentification sécurisé ainsi qu'une gestion des rôles afin d'adapter les droits d'accès aux différents profils d'utilisateurs.
- Déployer la solution dans un environnement conteneurisé avec Docker afin de faciliter sa portabilité, son déploiement et sa maintenance.

1.4 Périmètre fonctionnel du projet

1.4.1 Domaine des RH

Le périmètre fonctionnel du domaine RH repose sur quatre axes principaux, définis à partir des données sources de la Banque Extérieure d'Algérie. Le premier concerne la **gestion des effectifs**. Elle s'appuie sur l'historique du personnel, les affectations, les grades, les services ainsi que les types administratifs et techniques. Ces données permettent de suivre la structure organisationnelle et l'évolution des effectifs au fil du temps.

Le deuxième axe porte sur la **gestion des charges du personnel**. Il exploite les bulletins de paie, leurs éléments fixes, leurs différents types ainsi que les avantages liés à la possession d'un véhicule. L'objectif est d'analyser la masse salariale et d'étudier sa répartition selon différents critères.

Le troisième axe concerne la **gestion des crédits du personnel**. Elle prend en compte les prêts personnels accordés aux employés, leurs types et leurs lignes de remboursement. Ces informations permettent notamment de suivre l'évolution et l'encours des crédits accordés au personnel.

Enfin, le quatrième axe est consacré à la **gestion des absences**. Il regroupe les demandes de congé, leurs motifs et les soldes disponibles. Ces données permettent d'analyser les absences et de calculer des indicateurs tels que le taux d'absentéisme.

L'ensemble de ces données est intégré au Data Warehouse à travers un processus ETL. Elles servent ensuite à construire les principaux indicateurs décisionnels, notamment l'effectif, la masse salariale, le taux d'absentéisme et l'encours des crédits. Ces indicateurs sont présentés dans des dashboards interactifs qui offrent des filtres dynamiques et un accès sécurisé selon le rôle de l'utilisateur : RH, Finance ou Direction.

1.4.2 Domaine du Crédit

Le périmètre fonctionnel du domaine **Crédit** de la plateforme décisionnelle est consacré à l'analyse et au suivi des prêts personnels accordés aux employés de la Banque Extérieure d'Algérie. Il repose principalement sur les données issues des tables **pret_pers** (prêts personnels), **typ_pret** (types de prêts) et **lig_pret** (lignes de prêts).

Ce domaine permet d'exploiter les informations liées aux demandes de crédit, aux montants accordés, aux types de prêts, aux échéances ainsi qu'à l'état d'avancement des dossiers. Ces données offrent une vision globale des crédits accordés au personnel et facilitent leur suivi dans le temps.

L'objectif est de mettre à la disposition des responsables RH et financiers des indicateurs décisionnels pertinents, tels que le volume des crédits, leur répartition par type et leur évolution au fil des périodes. Ces indicateurs sont présentés à travers des tableaux de bord interactifs intégrant des filtres dynamiques, notamment par période, service et grade. La plateforme facilite ainsi le pilotage des crédits et le suivi des engagements de la banque envers son personnel.

1.4.3 Gestion des absences

Le périmètre fonctionnel de la **gestion des absences** s'intègre à la plateforme décisionnelle RH & Crédit de la Banque Extérieure d'Algérie. Il couvre les principales données liées aux demandes de congés, aux motifs d'absence et aux soldes de congés, issues respectivement des tables **dem_cng**, **motif_j** et **sold_cng**.

Cette composante permet d'intégrer les données dans le Data Warehouse à l'aide d'un processus ETL. Les indicateurs analytiques sont ensuite accessibles à travers une API REST sécurisée et présentés dans des tableaux de bord interactifs. Les utilisateurs autorisés peuvent ainsi consulter et analyser les données selon différents critères, notamment la période, le service ou le grade.

La plateforme facilite également le suivi des tendances d'absentéisme et des soldes de congés disponibles. Ces informations fournissent aux responsables RH, Finance et Direction une meilleure visibilité sur les absences et les congés du personnel, et contribuent ainsi à une prise de décision plus efficace.

1.5 Méthodologie de travail adoptée

La réalisation de ce projet repose sur une approche hybride qui combine la méthodologie de conception décisionnelle **Kimball** avec une démarche itérative inspirée de l'**Agile**. Cette approche permet de structurer la conception du système décisionnel tout en conservant une certaine flexibilité au cours du développement.

Dans un premier temps, une phase d'analyse de l'existant a été menée afin d'identifier les différentes sources de données RH et Crédit, notamment **personnel_hist**, **bulletin_h**, **pret_pers** et **dem_cng**. Cette étape a également permis de recueillir les besoins des utilisateurs finaux, principalement les responsables RH, Finance et Direction. À partir de ces éléments, la modélisation dimensionnelle a été réalisée selon un **schéma en étoile**, afin de définir les tables de faits et les tables de dimensions nécessaires à l'analyse des indicateurs.

Le processus **ETL** a ensuite été conçu et mis en œuvre avec **Talend Open Studio**. Il assure l'extraction des données depuis les sources, leur transformation et leur chargement dans le **Data Warehouse Oracle**. En parallèle, le développement de l'API backend a été réalisé avec **Spring Boot**, **Spring Data JPA** et **Spring Security**. L'intégration de **Keycloak** permet de gérer l'authentification et les rôles des utilisateurs.

Le frontend, dédié à la consultation des indicateurs, a été développé avec **Angular**, **Angular Material** et **ApexCharts**. Le développement a suivi une logique incrémentale afin de valider progressivement les différentes fonctionnalités, notamment la visualisation des indicateurs, les filtres dynamiques et la sécurité.

Enfin, l'ensemble de la solution a été conteneurisé à l'aide de **Docker et Docker Compose**. La documentation de l'API a été réalisée avec **Swagger/OpenAPI**, tandis que le code source et les différentes versions du projet ont été gérés avec **Git/GitLab**. Cette méthodologie a permis de maintenir une progression structurée tout en offrant la flexibilité nécessaire pour intégrer les ajustements apparus au cours du développement.

1.6 Conclusion

Ce premier chapitre a permis de présenter le contexte organisationnel et technique du projet. Après avoir présenté l'organisme d'accueil, **ARABSOFT**, nous avons exposé la problématique liée à l'exploitation fragmentée des données RH et Crédit de la Banque Extérieure d'Algérie. Les objectifs du stage ont ensuite été définis, ainsi que le périmètre fonctionnel du projet, qui couvre la gestion des effectifs, des charges du personnel, des crédits et des absences.

La méthodologie adoptée a également été présentée. Elle repose sur la combinaison de la démarche décisionnelle **Kimball** et d'une approche itérative inspirée de l'**Agile**, afin de structurer les différentes étapes du projet tout en conservant une certaine flexibilité.

Ces éléments constituent la base du travail réalisé. Les chapitres suivants présentent la conception et le développement de la plateforme décisionnelle, ainsi que les différentes étapes de sa mise en œuvre.

Chapitre 2 : État de l'art et concepts théoriques

2.1 Introduction à la Business Intelligence (BI)

La **Business Intelligence (BI)**, ou informatique décisionnelle, regroupe les méthodes et technologies permettant de transformer les données d'une organisation en informations utiles à l'analyse et à la prise de décision. Elle facilite notamment la production de rapports, d'indicateurs clés de performance (KPI) et de tableaux de bord.

Les données provenant de plusieurs systèmes peuvent être collectées, intégrées et organisées afin de permettre leur analyse selon différents critères. La BI permet ainsi de suivre l'évolution des activités, de comparer les résultats et d'identifier certaines tendances.

Le **Data Warehouse** constitue un élément central de cette architecture. Il regroupe les données provenant des systèmes opérationnels et les prépare pour leur exploitation analytique (Oracle, 2023).

Dans ce projet, la BI est appliquée aux domaines des **Ressources Humaines et du Crédit**. La plateforme permet notamment d'analyser les effectifs, les charges du personnel, les absences et les crédits accordés.

La chaîne générale peut être résumée comme suit :

Sources de données → Intégration → Data Warehouse → Analyse → Tableaux de bord

Data Warehouse Architecture

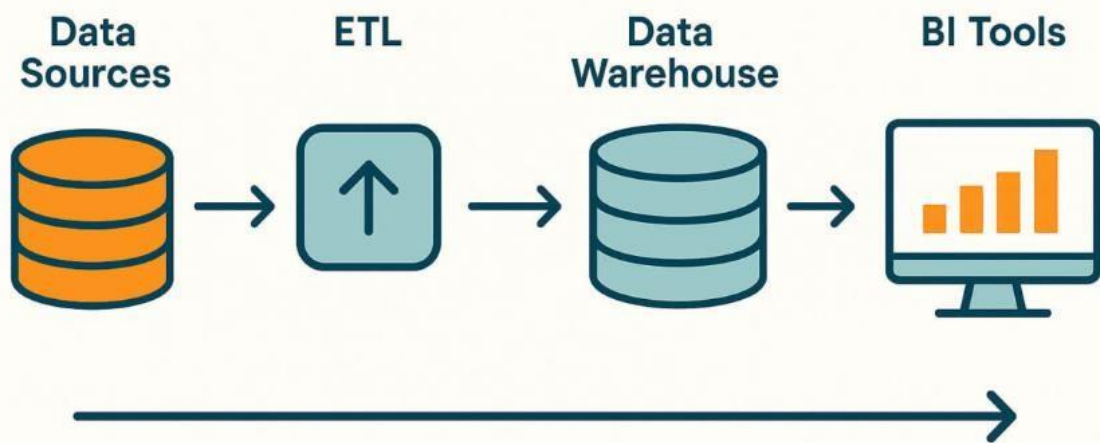


Figure 2 : Data Warehouse Architecture

2.2 Entrepôts de données (Data Warehouse) et modélisation décisionnelle

Un **Data Warehouse** est un système destiné principalement à l'analyse des données. Il centralise des informations provenant de différentes sources et conserve leur historique afin de faciliter les analyses dans le temps (Inmon, 2005 ; Oracle, 2023).

Contrairement aux bases opérationnelles, il est conçu pour répondre aux besoins analytiques et fournir des données cohérentes aux utilisateurs. Selon Inmon (2005), un entrepôt de données est notamment orienté sujet, intégré, historisé et relativement stable.

Dans ce projet, le Data Warehouse constitue l'intermédiaire entre les systèmes opérationnels de la **Banque Extérieure d'Algérie** et la plateforme de visualisation. Les données provenant d'Oracle sont préparées puis organisées pour l'analyse des domaines RH et Crédit.

La modélisation retenue s'appuie sur l'approche dimensionnelle de **Kimball**, qui organise les données autour de faits, de dimensions et d'une granularité définie pour chaque processus métier (Kimball & Ross, 2013).

2.2.1 Schéma en étoile

Le **schéma en étoile** est une forme de modélisation dimensionnelle dans laquelle une table de faits centrale est reliée à plusieurs tables de dimensions. Les faits regroupent les mesures à analyser, tandis que les dimensions fournissent le contexte nécessaire à leur interprétation (Kimball & Ross, 2013).

Cette organisation facilite les analyses selon plusieurs axes. Par exemple, les effectifs peuvent être étudiés selon le temps, le service ou le grade, tandis que les crédits peuvent être analysés selon la période, le service ou le type de prêt.

Dans ce projet, le schéma en étoile est utilisé pour structurer les données relatives aux **Ressources Humaines, aux charges, aux absences et aux crédits**.

La conception repose principalement sur l'identification du processus métier, la définition de sa granularité et la détermination des faits et dimensions nécessaires (Kimball & Ross, 2013).

2.2.2 Tables de faits et dimensions

La **table de faits** représente le processus métier étudié. Elle contient principalement les mesures ainsi que les références vers les dimensions associées (Kimball & Ross, 2013).

Dans ce projet, les faits concernent notamment :

La **table de faits** représente le processus métier étudié. Elle contient principalement les mesures ainsi que les références vers les dimensions associées (Kimball & Ross, 2013).

Dans ce projet, les faits concernent notamment :

- les effectifs du personnel ;
- les charges et montants associés au personnel ;
- les crédits accordés ;
- les absences et leur durée.

Les **dimensions** permettent d'analyser ces mesures selon différents axes :

- Dimension Temps** : analyse par année, trimestre ou mois ;
- Dimension Personnel** : informations relatives aux employés ;
- Dimension Service** : analyse selon les services ;
- Dimension Grade** : analyse selon les catégories professionnelles ;
- **Dimension Type de prêt** : classification des crédits ;
- Dimension Motif d'absence** : analyse des absences selon leur cause.

L'association des faits et des dimensions permet ainsi de réaliser des analyses multidimensionnelles adaptées aux besoins de la plateforme (Kimball & Ross, 2013).

2.2.3 Organisation du schéma décisionnel

Les dimensions représentent les axes d'analyse, tandis que les faits contiennent les mesures utilisées pour calculer les indicateurs.

Pour les Ressources Humaines, une analyse peut suivre le chemin :

Temps → Service → Grade → Effectif

Pour le domaine Crédit, elle peut être organisée selon :

Temps → Service → Type de prêt → Montant du crédit

Cette organisation permet de croiser plusieurs dimensions et de produire des indicateurs adaptés aux besoins des utilisateurs.

2.2.4 Avantages du schéma en étoile

Le schéma en étoile présente une structure simple qui facilite la compréhension du modèle et la construction des requêtes analytiques.

La séparation entre les **faits** et les **dimensions** permet de distinguer les mesures à analyser du contexte dans lequel elles doivent être étudiées. Cette organisation facilite également les opérations de filtrage et d'agrégation (Kimball & Ross, 2013).

Dans les environnements décisionnels, cette approche est notamment utilisée pour simplifier les analyses et organiser les données destinées au reporting (Microsoft, 2026).

Dans le cadre du projet, elle permet de disposer d'un modèle clair pour l'exploitation des données RH et Crédit.

2.3 Processus ETL (Extract – Transform – Load)

Le processus **ETL** (*Extract, Transform, Load*) regroupe les opérations nécessaires pour extraire les données des systèmes sources, les transformer puis les charger dans le Data Warehouse.

Il comprend trois étapes principales :

1. **Extraction** : récupération des données ;
2. **Transformation** : nettoyage, contrôle et préparation ;
3. **Chargement** : intégration dans le Data Warehouse.

Cette chaîne permet de préparer les données avant leur utilisation dans les analyses décisionnelles (Oracle, 2023).

Dans ce projet, les données provenant principalement d'**Oracle** sont traitées avec **Talend Open Studio** avant leur chargement dans le Data Warehouse.

Le flux général est le suivant :

Sources Oracle → Extraction → Transformation → Chargement → Data Warehouse → Tableaux de bord

2.3.1 Extraction

L'extraction consiste à récupérer les données nécessaires depuis les systèmes sources.

Dans le projet, les données sont principalement extraites d'Oracle à partir des tables relatives au personnel, aux affectations, aux grades, aux services, aux bulletins de paie, aux prêts et aux congés.

Cette étape permet de sélectionner les informations nécessaires au modèle décisionnel tout en préservant le fonctionnement des systèmes opérationnels.

2.3.2 Transformation

La transformation prépare les données extraites avant leur intégration dans le Data Warehouse. Elle permet notamment d'harmoniser les formats et de corriger certaines incohérences.

Les principales opérations comprennent :

- ☐ le traitement des valeurs manquantes ;
- ☐ le contrôle de la cohérence des données ;
- ☐ l'harmonisation des formats ;
- ☐ la réalisation des jointures ;
- ☐ la sélection des données utiles ;
- ☐ la préparation des faits et des dimensions.

Cette étape contribue directement à la qualité des données utilisées pour calculer les indicateurs.

2.3.3 Chargement

Le chargement consiste à intégrer les données transformées dans les tables du Data Warehouse, tout en respectant les relations entre les faits et les dimensions.

Dans le projet, les données préparées avec **Talend Open Studio** sont chargées dans les structures décisionnelles dédiées aux domaines RH et Crédit. Elles peuvent ensuite être exploitées par la plateforme pour produire les différents indicateurs.

Le processus ETL permet ainsi d'assurer une séparation entre les systèmes opérationnels et l'environnement analytique (Oracle, 2023).

2.4 Architectures des applications web modernes

Les applications web modernes séparent généralement les responsabilités liées à l'interface, à la logique applicative et à l'accès aux données.

Dans ce projet, l'architecture repose sur un **frontend Angular** et un **backend Java Spring Boot**. Angular assure la présentation et les interactions avec l'utilisateur, tandis que Spring Boot prend en charge les traitements et l'accès au Data Warehouse.

La communication entre les deux composants est assurée par une **API REST**.

L'architecture peut être représentée comme suit :

Utilisateur → Angular → API REST Spring Boot → Data Warehouse Oracle

Cette séparation facilite la maintenance et l'évolution des différents composants.

2.4.1 API REST

REST (*Representational State Transfer*) est un style architectural proposé par Roy Fielding pour les applications distribuées (Fielding, 2000).

Une API REST permet aux composants d'une application de communiquer via HTTP. Les principales méthodes utilisées sont :

- ☐ GET : consultation ;
- ☐ POST : création ;
- ☐ PUT : modification ;
- ☐ DELETE : suppression.

Dans la plateforme, les requêtes **GET** sont principalement utilisées pour récupérer les données nécessaires aux tableaux de bord.

Le backend Spring Boot est organisé en plusieurs couches :

- ☐ Controller : réception des requêtes HTTP ;
- ☐ Service : logique applicative ;
- ☐ Repository : accès aux données ;
- ☐ Data **Warehouse** : stockage des données décisionnelles.

Les échanges entre Angular et Spring Boot utilisent le format **JSON**.

Le flux principal est donc :

Angular → **Controller** → **Service** → **Repository** → **Data Warehouse**

La sécurité de l'API est assurée par **Spring Security**, associé à Keycloak pour la gestion de l'authentification et des rôles.

2.4.2 Applications SPA (Single Page Application)

Une **Single Page Application (SPA)** charge principalement une seule page dans le navigateur et actualise son contenu de manière dynamique lors de la navigation.

Angular permet de développer ce type d'application et fournit notamment un système de routage permettant de changer les vues sans rechargement complet de la page (Angular, 2026).

Dans ce projet, Angular constitue la couche frontend. Il permet d'afficher les indicateurs, graphiques et tableaux de bord et communique avec le backend Spring Boot à travers l'API REST.

Cette architecture offre ainsi une interface dynamique tout en séparant la présentation de la logique métier.

2.5 Sécurité et gestion des identités

La sécurité constitue un élément important de la plateforme en raison de la nature des données RH, financières et liées aux crédits.

La solution combine plusieurs technologies :

- OAuth **2.0** pour l'autorisation ;
- OpenID **Connect** pour l'authentification et l'identité ;
- Keycloak pour la gestion centralisée des utilisateurs et des rôles ;
- Spring **Security** pour protéger l'API.

Cette architecture permet de centraliser les mécanismes de sécurité et de contrôler l'accès aux ressources selon le profil de l'utilisateur.

2.5.1 OAuth 2.0 et OpenID Connect

OAuth 2.0 est un cadre d'autorisation permettant à une application d'accéder à des ressources protégées à l'aide de jetons d'accès (Hardt, 2012).

OpenID Connect (OIDC) complète OAuth 2.0 en ajoutant une couche dédiée à l'identité de l'utilisateur. Il permet notamment à l'application de vérifier l'identité de l'utilisateur authentifié (Sakimura et al., 2023).

Pour une application SPA, le flux **Authorization Code avec PKCE** permet de renforcer la protection du code d'autorisation. PKCE a notamment été conçu pour réduire les risques liés à l'interception de ce code pour les clients publics (Sakimura et al., 2015).

Dans le projet, le processus peut être résumé ainsi :

**Utilisateur → Angular → Keycloak → Code d'autorisation →
Jeton d'accès → API Spring Boot**

Le jeton est transmis à l'API avec l'en-tête :

Authorization: Bearer <access_token>

Spring Security vérifie ensuite le jeton et les autorisations associées avant de permettre l'accès aux ressources.

Les principaux rôles définis dans la plateforme sont :

- ☐ **ROLE_RH ;**
- ☐ **ROLE_FINANCE ;**
- ☐ **ROLE_DIRECTION.**

2.5.2 Keycloak

Keycloak est une solution open source de gestion des identités et des accès. Elle prend notamment en charge OAuth 2.0 et OpenID Connect et permet de centraliser la gestion des utilisateurs, des rôles et des clients.

Dans ce projet, Keycloak joue le rôle de **fournisseur d'identité et de serveur d'autorisation**. Il assure l'authentification des utilisateurs et fournit les informations nécessaires à la gestion des accès.

Les rôles principaux sont :

- ☐ RH ;
- ☐ Finance ;
- ☐ Direction.

L'application Angular utilise Keycloak pour l'authentification, tandis que l'API Spring Boot vérifie les jetons reçus avant d'autoriser l'accès aux ressources.

Cette séparation permet de distinguer clairement **l'authentification, l'autorisation et la logique métier**.

2.6 Conclusion

Ce chapitre a présenté les principaux concepts théoriques et technologiques liés à la plateforme décisionnelle RH & Crédit.

La **Business Intelligence** et le **Data Warehouse** ont d'abord été étudiés afin de présenter les principes de l'analyse décisionnelle. La modélisation dimensionnelle et le **schéma en étoile** ont ensuite permis de définir l'organisation des faits et des dimensions utilisées dans le projet.

Le processus **ETL** a également été présenté à travers les étapes d'extraction, de transformation et de chargement des données provenant d'Oracle vers le Data Warehouse à l'aide de Talend.

Le chapitre a ensuite abordé l'architecture web de la solution, basée sur **Angular, Spring Boot et une API REST**, ainsi que le fonctionnement des applications SPA.

Enfin, les mécanismes de sécurité reposant sur **OAuth 2.0, OpenID Connect, PKCE, Keycloak et Spring Security** ont été présentés afin d'expliquer la gestion de l'authentification et des droits d'accès.

Ces concepts constituent le socle théorique et technologique nécessaire à la compréhension de la solution développée. Le chapitre suivant sera consacré à **l'analyse et à la spécification des besoins**.

Chapitre 3 : Analyse et spécification des besoin

3.1 Étude de l'existant

Actuellement, la **Banque Extérieure d'Algérie (BEA)** dispose de plusieurs systèmes d'information métiers qui gèrent séparément les données relatives aux ressources humaines, notamment le personnel, les affectations, les grades et les congés, ainsi que les données liées aux crédits, telles que les prêts et les échéances. Ces informations sont principalement stockées dans une base de données **Oracle**.

Cependant, ces systèmes reposent essentiellement sur une architecture **OLTP**, conçue pour répondre aux besoins de gestion opérationnelle quotidienne. Ils ne fournissent pas directement un environnement adapté à l'analyse décisionnelle et au suivi global des indicateurs.

L'exploitation des données RH et Crédit repose alors sur des requêtes ponctuelles et des traitements qui peuvent nécessiter des interventions manuelles. Cette situation augmente le temps consacré à la préparation des informations et peut également entraîner des erreurs lors de leur traitement ou de leur consolidation. Par ailleurs, l'absence d'une vision centralisée rend plus difficile le suivi global des indicateurs par les responsables RH et financiers.

L'absence d'outils de visualisation interactifs constitue également une limite. Les utilisateurs disposent de moyens réduits pour explorer les données, comparer les résultats selon différents critères ou suivre facilement l'évolution des indicateurs tels que les effectifs, les charges du personnel, les absences et les encours de crédit.

Face à ces limites, il devient nécessaire de mettre en place une **plateforme décisionnelle centralisée** capable de regrouper les données provenant des différents systèmes sources, de les transformer et de les structurer dans un **Data Warehouse**. La solution doit également permettre de restituer les informations sous forme de tableaux de bord interactifs, afin de faciliter l'analyse des indicateurs et d'apporter aux responsables des informations fiables et adaptées à leurs besoins décisionnels.

3.2 Identification des acteurs et de leurs rôles

L'identification des **acteurs** constitue une étape essentielle de l'analyse des besoins. Elle permet de déterminer les différents utilisateurs de la plateforme et de définir leurs interactions avec le système. Dans le cadre de ce projet, quatre acteurs principaux ont été identifiés.

Le **service des RH (RH)** consulte et analyse les indicateurs relatifs aux effectifs, aux absences et aux charges du personnel. Le **service financier (Finance)** accède principalement aux données liées aux crédits ainsi qu'aux différents indicateurs financiers concernant les employés.

La **Direction** dispose d'une vue globale sur les indicateurs RH et Crédit. Cette vision lui permet de suivre les principales évolutions, d'appuyer la prise de décision et de contribuer à la planification stratégique.

Enfin, les **administrateurs** assurent la gestion des utilisateurs, des rôles et des droits d'accès. Ils interviennent également dans la configuration et l'administration de la plateforme d'authentification à l'aide de **Keycloak**.

Chaque acteur possède un rôle spécifique auquel sont associés des droits et des permissions. Cette organisation garantit un accès adapté aux besoins de chaque profil et respecte le **principe du moindre privilège**, en limitant les autorisations aux seules ressources nécessaires à l'utilisateur.

3.3 Spécification des besoins fonctionnels

La plateforme décisionnelle doit répondre à plusieurs besoins fonctionnels couvrant les domaines des **ressources humaines (RH)** et du **crédit**. Elle doit tout d'abord assurer une authentification sécurisée des utilisateurs et une gestion des accès fondée sur les rôles. Les profils **RH, Finance et Direction** disposent ainsi de droits et de vues adaptés à leurs responsabilités.

Le système doit permettre de consulter les principaux **indicateurs clés de performance (KPI)** relatifs aux effectifs, aux charges du personnel, aux absences et aux crédits accordés aux employés. Ces informations sont présentées à travers des tableaux de bord interactifs, permettant aux utilisateurs de suivre les indicateurs et leur évolution à partir des données disponibles.

La plateforme doit également proposer des **filtres dynamiques**, notamment par période, service et grade. Cette fonctionnalité permet aux utilisateurs d'affiner les résultats et d'adapter les analyses à leurs besoins.

Par ailleurs, le système doit assurer l'intégration des données provenant des systèmes sources à travers un processus **ETL**. Celui-ci permet d'extraire, de transformer et de charger les données dans le **Data Warehouse**, afin de garantir leur cohérence et de faciliter leur exploitation analytique.

Enfin, la solution doit exposer une **API REST sécurisée** pour fournir un accès standardisé aux données analytiques. Elle doit également proposer une interface web ergonomique et responsive, facilitant la consultation des indicateurs, des rapports et des visualisations par les différents utilisateurs métier.

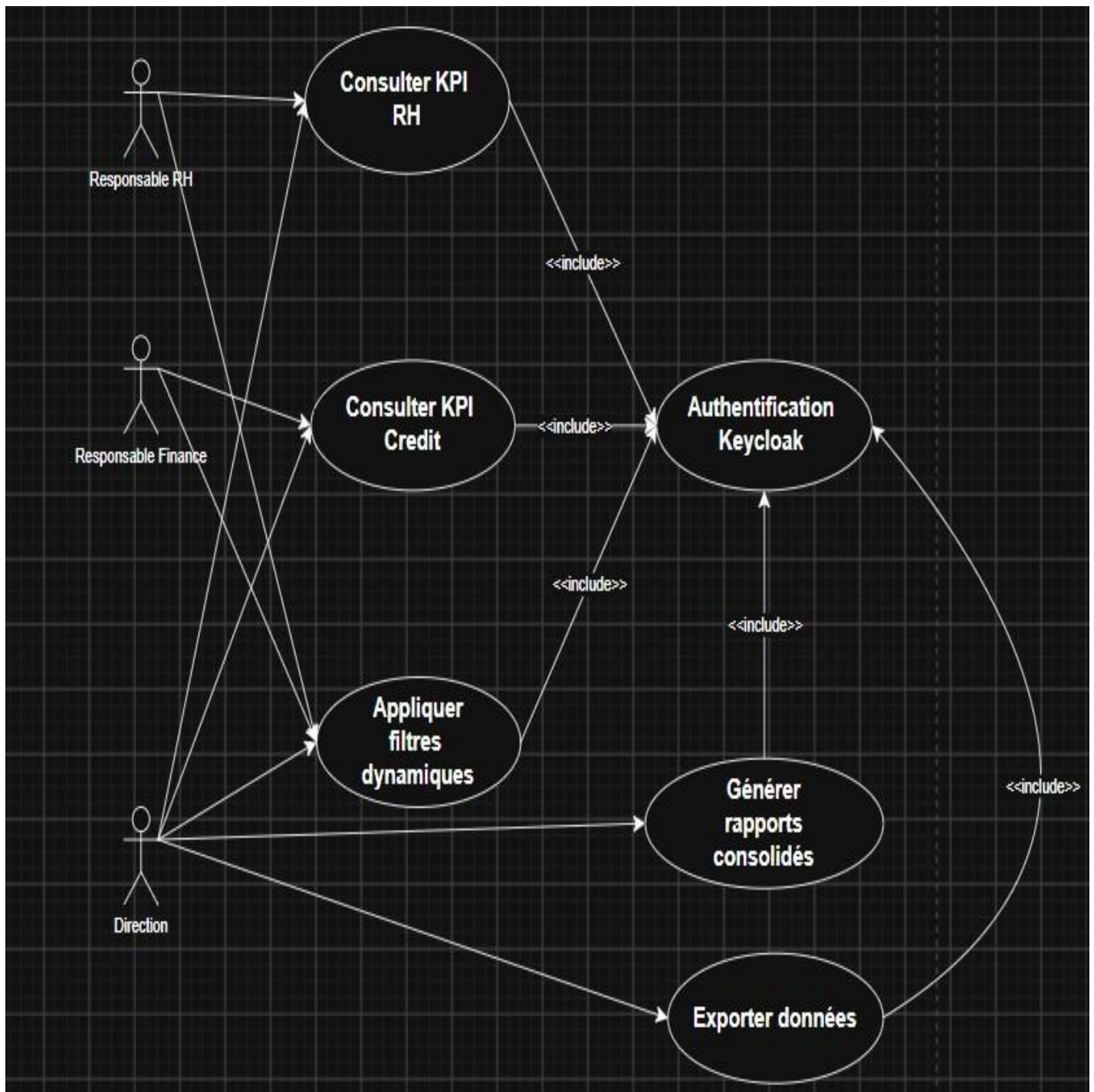


Figure 3 : Diagramme des cas d'utilisation de la plateforme décisionnelle

3.4 Spécification des besoins non fonctionnels

Au-delà des fonctionnalités métier, la plateforme doit respecter plusieurs **exigences non fonctionnelles** afin de garantir sa qualité, sa fiabilité et son adéquation avec le contexte bancaire.

En matière de **sécurité**, l'application doit assurer une authentification robuste ainsi qu'une gestion précise des rôles à l'aide de **Keycloak**. Les profils RH, Finance et Direction disposent ainsi de droits adaptés à leurs responsabilités. Les échanges contenant des données sensibles doivent également être protégés par des mécanismes de chiffrement tels que **HTTPS/TLS**, afin de renforcer la confidentialité des informations.

La **performance** constitue également une exigence importante. Les tableaux de bord et les requêtes analytiques doivent fournir les résultats dans des délais raisonnables, même lorsque le Data Warehouse contient un volume important de données historiques. Cette contrainte nécessite notamment une modélisation décisionnelle adaptée, basée sur le **schéma en étoile**, ainsi que des processus ETL correctement optimisés.

La plateforme doit par ailleurs garantir une bonne **fiabilité** des traitements. Les processus ETL doivent assurer une alimentation cohérente du Data Warehouse et limiter les risques d'erreurs ou de perte de données. Une disponibilité satisfaisante du système est également nécessaire pour permettre aux utilisateurs d'accéder aux indicateurs lorsqu'ils en ont besoin.

La **scalabilité** et la portabilité sont prises en compte grâce à l'utilisation de **Docker et Docker Compose**. La conteneurisation facilite le déploiement de la solution et permet d'adapter plus facilement son environnement lorsque les besoins évoluent.

L'**ergonomie** constitue également un aspect important. L'interface Angular doit rester claire, intuitive et responsive afin de permettre aux utilisateurs métier, y compris ceux qui ne disposent pas de compétences techniques particulières, de consulter facilement les indicateurs et les tableaux de bord.

Enfin, la solution doit être **maintenable et extensible**. Son architecture doit permettre l'ajout de nouveaux indicateurs, de nouvelles fonctionnalités ou de nouveaux modules sans remettre en cause l'organisation générale du système. Une documentation

technique claire, notamment grâce à **Swagger/OpenAPI** pour l'API REST, facilite également la maintenance, la reprise du projet et son évolution future.

3.5 Diagramme de cas d'utilisation

Le diagramme de cas d'utilisation de la **plateforme décisionnelle RH & Crédit** présente les principales interactions entre le système et ses quatre acteurs. Il permet d'identifier les fonctionnalités accessibles à chaque profil ainsi que les règles qui régissent leur accès.

Le responsable **RH** dispose d'un tableau de bord dédié aux indicateurs liés aux ressources humaines. Il peut consulter les principaux KPI, appliquer des filtres selon différents critères, tels que la période, le service ou le grade, et analyser les données relatives aux charges du personnel et aux absences.

L'acteur **Finance** accède principalement au tableau de bord consacré au Crédit. Il peut consulter les indicateurs liés aux prêts accordés au personnel, appliquer des filtres dynamiques pour affiner les analyses et générer des rapports financiers à partir des données disponibles.

La **Direction** dispose d'une vision globale de la plateforme. Elle peut consulter les tableaux de bord RH et Crédit, analyser les différents indicateurs et générer des rapports consolidés. Des fonctionnalités d'export lui permettent également d'extraire certaines données afin de faciliter le reporting et le suivi des activités.

L'**Administrateur** est chargé de la gestion et de la supervision de la plateforme. Il assure notamment la gestion des utilisateurs, l'attribution des rôles et le contrôle des droits d'accès au système.

Enfin, tous les acteurs doivent s'**authentifier** avant d'accéder aux fonctionnalités de la plateforme. Cette authentification est gérée par **Keycloak**, qui assure également la gestion des rôles et des autorisations. Les accès sont ainsi contrôlés en fonction du profil de chaque utilisateur, ce qui permet de maintenir une séparation claire entre les droits des rôles **RH, Finance, Direction et Administrateur**.

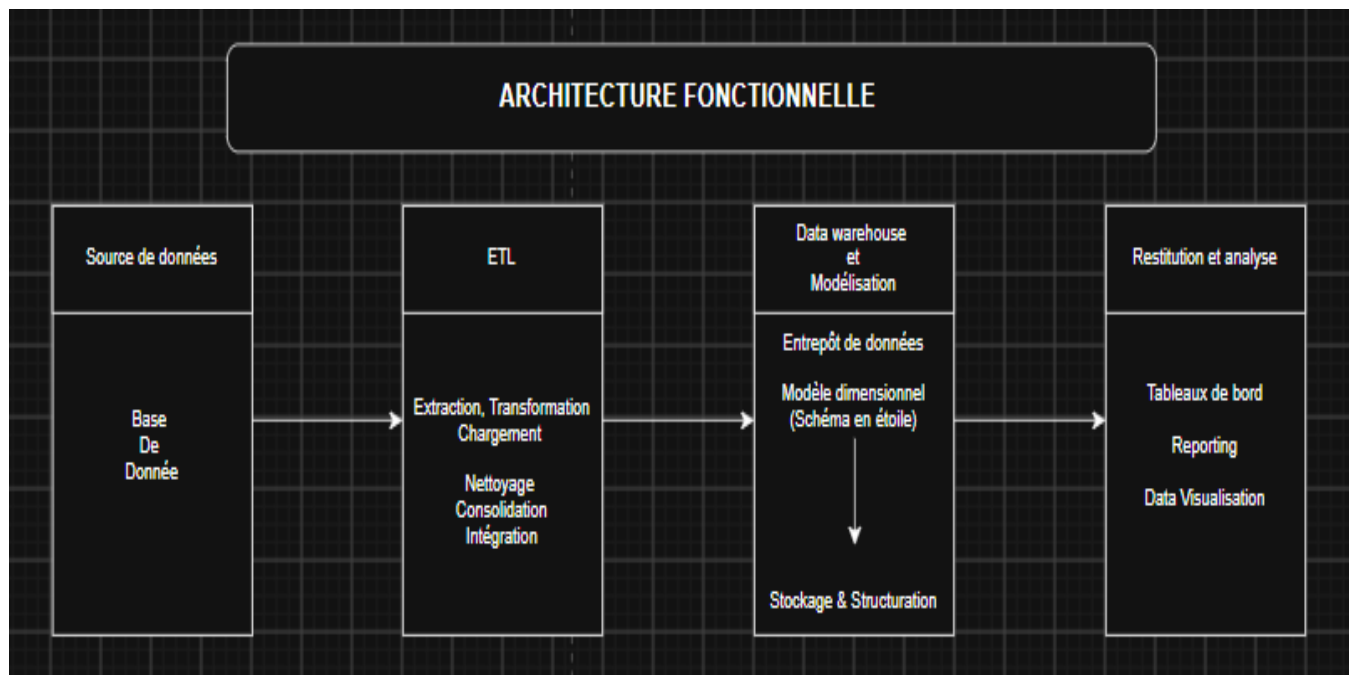


Figure 4 : Architecture fonctionnelle

3.6 Conclusion

Ce chapitre a été consacré à **l'analyse et à la spécification des besoins** du projet, une étape essentielle avant d'entamer la phase de conception. L'étude de l'existant a permis d'identifier les principales limites liées au traitement et à l'analyse des données RH et Crédit. Ces constats ont ainsi mis en évidence la nécessité de mettre en place une solution décisionnelle intégrée et adaptée aux besoins des utilisateurs.

Les principaux acteurs de la plateforme ont ensuite été identifiés, ainsi que leurs rôles et leurs besoins respectifs. Il s'agit notamment des responsables **RH**, **Finance** et de la **Direction**, auxquels s'ajoute le profil **Administrateur** pour la gestion des utilisateurs et des accès. Chaque profil dispose de droits adaptés à ses responsabilités.

La spécification des besoins fonctionnels et non fonctionnels a permis de définir les principales fonctionnalités attendues, telles que la consultation des indicateurs, l'utilisation de filtres dynamiques, la génération de rapports et la gestion sécurisée des accès. Elle a également permis de préciser les exigences liées à la **performance**, à la **sécurité**, à l'**ergonomie**, à la **fiabilité** et à la **maintenabilité** de la solution.

Enfin, les diagrammes de cas d'utilisation ont permis de représenter graphiquement les principales interactions entre les acteurs et le système. Ils offrent ainsi une vision globale des fonctionnalités et des règles d'accès définies pour la plateforme.

L'ensemble de ces éléments constitue une base solide pour la phase de conception. Le chapitre suivant présentera donc l'architecture technique, la modélisation décisionnelle et les différents choix retenus pour transformer les besoins identifiés en une solution concrète répondant aux objectifs de la plateforme décisionnelle RH & Crédit.

Chapitre 4 : Conception de la plateforme

Décisionnelle

4.1 Analyse des sources de données

Dans le cadre de la mise en place de la plateforme décisionnelle pour la **Banque Extérieure d'Algérie (BEA)**, l'analyse des sources de données constitue une première étape essentielle de la conception du **Data Warehouse** et du processus **ETL**. Les données opérationnelles de l'établissement sont hébergées dans un système de gestion de bases de données **Oracle**.

L'étude du système source a permis d'identifier quatre principaux domaines fonctionnels nécessaires à la construction du Data Warehouse :

- **Gestion des effectifs** : ce domaine regroupe les données relatives à la structure organisationnelle et au parcours des employés. Il s'appuie notamment sur les tables **personnel**, **Hist**, **affectation**, **grade**, **service** et **typ_adm_tech**.
- **Charges du personnel** : ce domaine concerne les données liées à la paie et aux charges salariales. Il exploite notamment les tables **bulletin_h**, **par_fixe**, **Type_bulletin** et **Possede_vh**.
- **Gestion des absences et des congés** : ce domaine permet de suivre les absences, leurs différents motifs ainsi que les informations relatives aux congés. Il repose principalement sur les tables **Dem_cng**, **motif_j** et **sold_cng**.
- **Gestion des crédits** : cet axe regroupe les informations relatives aux prêts accordés au personnel et à leurs caractéristiques. Les principales tables concernées sont **pret_pers**, **typ_pret** et **Lig_pret**.

L'analyse de ces structures transactionnelles permet de mieux comprendre les relations entre les différentes données et d'identifier

les informations nécessaires à l'analyse décisionnelle. Elle contribue également à évaluer leur qualité et leur cohérence, puis à définir les règles de transformation nécessaires avant leur intégration dans le **modèle dimensionnel en schéma en étoile**.

4.2 Conception du Data Warehouse

La modélisation décisionnelle retenue repose sur quatre schémas en étoile, correspondant respectivement aux effectifs, aux charges du personnel, aux crédits et aux absences. Cette organisation permet d'analyser chaque domaine de manière indépendante tout en partageant certaines dimensions communes, notamment Dim Personnel.

1. Schéma en étoile – Fait Effectif

La table de faits Fait Effectif stocke les mesures relatives à l'affectation des employés.

- ☐ **Granularité** : un employé affecté à un service et un grade à une date donnée.
- ☐ **Mesures** : effectif.
- ☐ **Dimensions** : Dim Personnel, Dim Service, Dim Grade, Dim_CATEG, Dim Affectation, Dim_TypAdmTech.

Ce schéma permet d'analyser l'évolution et la répartition des effectifs selon le service, le grade et le type d'employé.

2. Schéma en étoile – Fait ChargesPersonnel

La table de faits Fait Charges Personnel regroupe les informations de masse salariale.

- ☐ **Granularité** : un élément de rémunération d'un employé pour une période donnée.
- ☐ **Mesures** : montants des charges, salaires et avantages.
- ☐ **Dimensions** : Dim Personnel, Dim Bulletin, Dim TypeBulletin, Dim ParFixe, Dim PossedeVehicule.

Il permet d'analyser la masse salariale selon le type de bulletin, la période et les avantages accordés.

3. Schéma en étoile – Fait Crédit

La table de faits Fait Crédit centralise les données relatives aux prêts personnels.

- ☐ **Granularité** : un prêt ou une ligne de prêt.
- ☐ **Mesures** : montant du crédit, encours.
- ☐ **Dimensions** : Dim Personnel, Dim TypePret, Dim LignePret, Dim PretPers.

Ce schéma permet de suivre l'encours des crédits et leur répartition par type de prêt.

4. Schéma en étoile – Fait Absences

La table de faits Fait Absences regroupe les données d'absences et de congés.

- ☐ **Granularité** : une demande de congé ou une absence.
- ☐ **Mesures** : durée d'absence, solde de congés.
- ☐ Dimensions : Dim Personnel, Dim DemandeCongé, Dim MotifAbsence, Dim SoldeConge.
- Il permet de calculer le taux d'absentéisme et d'analyser les motifs d'absence.

Figure 5 : Schémas en étoile du Data Warehouse RH et Crédit

Schéma en étoile — Fait_Effectif

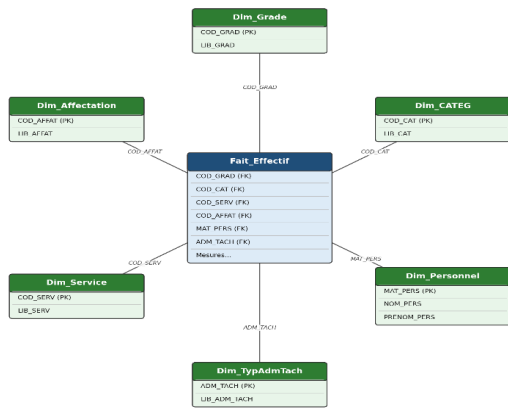


Schéma en étoile — Fait_ChargesPersonnel

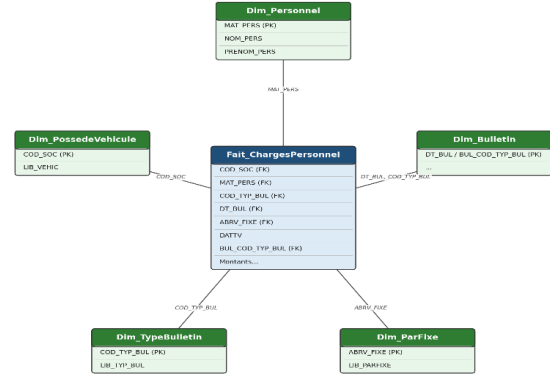


Schéma en étoile — Fait_Credit

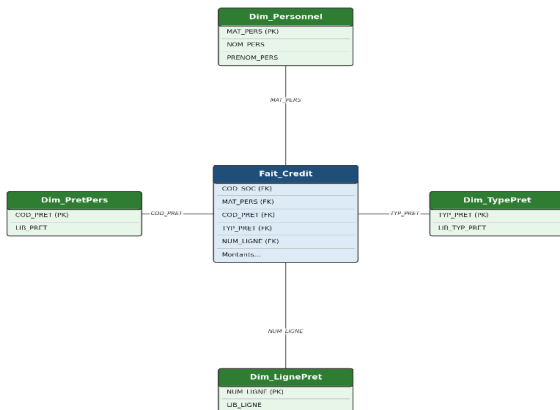
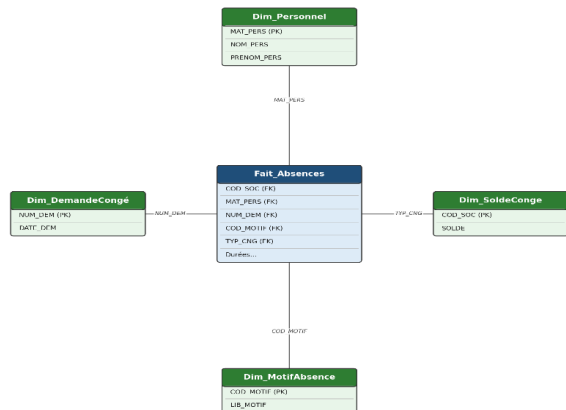


Schéma en étoile — Fait_Absences



4.3 Conception du processus ETL

La conception du processus **ETL** (*Extract, Transform, Load*) constitue un élément essentiel de la plateforme décisionnelle. Elle assure l'intégration des données provenant des systèmes d'information RH et des services financiers, tout en préparant ces données à leur exploitation analytique.

Les données sont d'abord extraites des systèmes opérationnels **Oracle**, notamment à partir des tables **personnelles_hist**, **bulletin_h**, **pret_pers**, **Dem_cng**, etc. Elles sont ensuite transformées afin d'améliorer leur qualité et leur cohérence. Cette étape comprend différentes opérations, telles que le nettoyage des données, l'harmonisation des formats et leur mise en correspondance avec les dimensions définies dans le modèle décisionnel.

Une fois préparées, les données sont chargées dans les différentes tables de faits et de dimensions du **Data Warehouse**. Le chargement respecte les relations entre les tables ainsi que les contraintes d'intégrité référentielle.

Le processus est réalisé avec **Talend Open Studio**, qui permet d'automatiser les différentes étapes du flux de données et de faciliter l'alimentation régulière du Data Warehouse. Cette organisation contribue à maintenir des données cohérentes et exploitables pour les traitements analytiques.

Le processus ETL constitue ainsi un maillon essentiel de la plateforme décisionnelle. Il fournit les données nécessaires au calcul des principaux indicateurs, tels que l'évolution des effectifs, la masse salariale, l'encours des crédits et le taux d'absentéisme. Ces indicateurs peuvent ensuite être exploités dans les tableaux de bord interactifs afin de faciliter le suivi et l'analyse des activités de la banque.

Processus ETL détaillé – Talend Open Studio

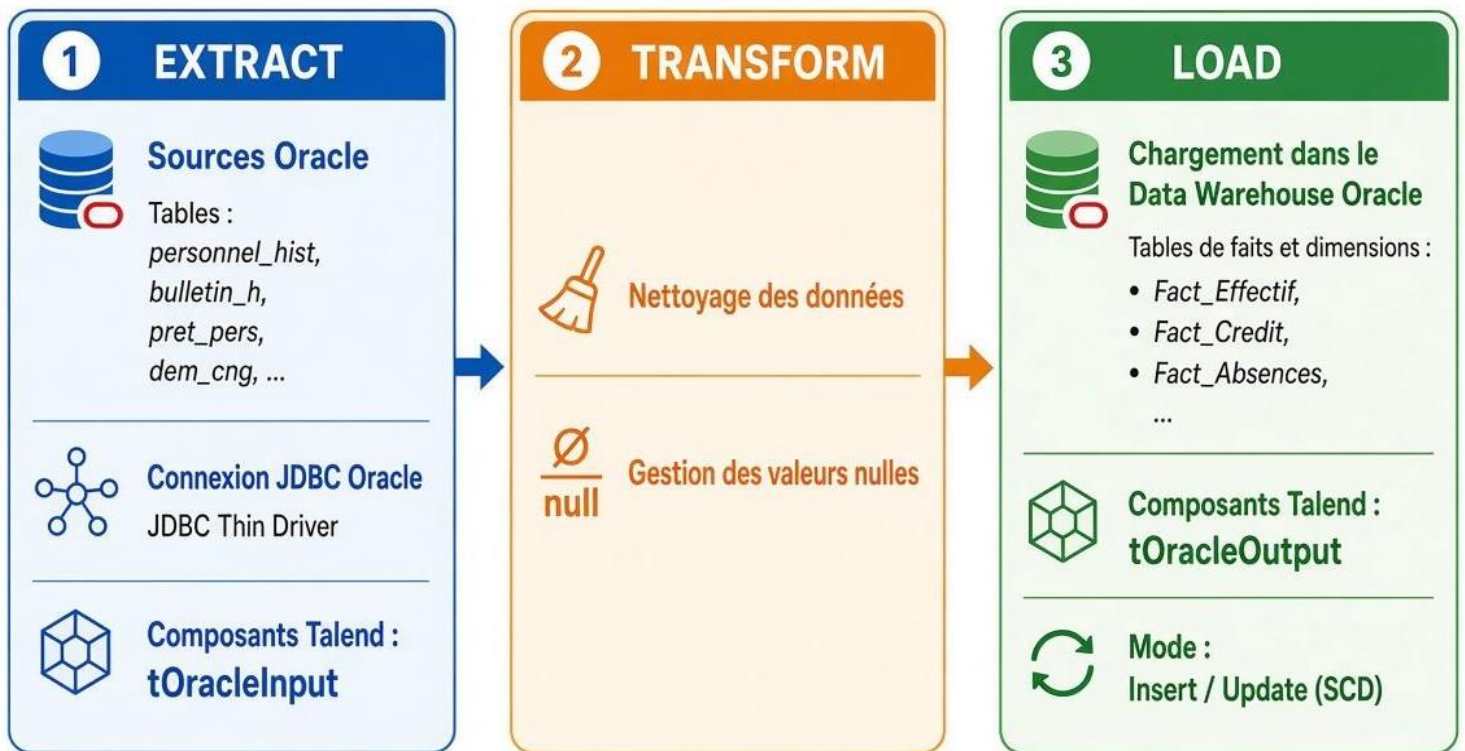


Figure 6 : Processus ETL détaillé

4.4 Conception des indicateurs de performance (KPIs RH et Crédit)

La définition des **KPI** constitue une étape importante dans la conception de la plateforme décisionnelle. Elle consiste à sélectionner des indicateurs pertinents permettant de mesurer, de suivre et d'analyser les activités liées aux ressources humaines et au crédit.

Pour le domaine **RH**, les principaux indicateurs retenus concernent notamment l'évolution des effectifs, leur répartition par service et par grade, le suivi de l'absentéisme ainsi que l'analyse des charges du personnel. Ces indicateurs permettent d'obtenir une vision synthétique de la situation des ressources humaines et de son évolution au fil du temps.

Pour le domaine **Crédit**, les indicateurs permettent principalement de suivre les montants des prêts accordés, leur évolution dans le temps et leur répartition selon les différents types de prêts. Ils offrent ainsi une meilleure visibilité sur les crédits accordés au personnel.

Les KPI sont calculés à partir des données intégrées dans le **Data Warehouse**. Ils peuvent être analysés selon plusieurs dimensions, notamment la période, le service, le grade ou le type de prêt. Les résultats sont ensuite présentés sous forme de **tableaux de bord interactifs**, permettant aux responsables RH et financiers de consulter les indicateurs, d'affiner leurs analyses et de disposer d'informations utiles à la prise de décision.

4.5 Conclusion

Dans ce chapitre, nous avons présenté la conception de la plateforme décisionnelle dédiée à l'analyse des données RH et Crédit. Nous avons tout d'abord étudié les différentes sources de données afin de comprendre leur organisation et d'identifier les informations nécessaires aux analyses.

Nous avons ensuite conçu le **Data Warehouse** en adoptant une structure adaptée aux besoins décisionnels du projet. Le processus **ETL** a également été défini afin d'assurer l'extraction, la transformation et le chargement des données dans l'entrepôt de données, tout en garantissant leur cohérence et leur qualité.

Enfin, les principaux **KPI RH et Crédit** ont été définis afin de suivre les indicateurs retenus et de faciliter leur analyse selon différents axes. Ces éléments constituent la base technique et fonctionnelle de la solution.

Le chapitre suivant sera consacré à **l'implémentation et à la restitution des résultats**. Il présentera la mise en œuvre des différents composants de la plateforme ainsi que les tableaux de bord développés pour visualiser et analyser les indicateurs décisionnels.

Chapitre 5 : Conception de l'architecture applicative

5.1 Architecture globale de la solution

L'**architecture globale de la solution** repose sur une approche modulaire et en couches. Elle permet d'assurer un flux cohérent et sécurisé des données, depuis les systèmes sources jusqu'à leur restitution sous forme de tableaux de bord. Chaque couche possède un rôle précis et communique avec les autres à travers des interfaces définies.

Les données provenant de la base **Oracle** sont d'abord extraites et traitées à l'aide des processus **ETL**. Elles sont ensuite chargées dans le **Data Warehouse**, où elles sont organisées selon une modélisation en **schéma en étoile**. Cette couche décisionnelle constitue la base du calcul et de l'analyse des différents KPI relatifs aux RH et au Crédit.

Les données analytiques sont ensuite exposées à travers une **API REST développée avec Spring Boot**. Cette API assure la communication entre le Data Warehouse et l'application web, tout en permettant de centraliser les traitements nécessaires à l'accès aux données.

Le **frontend**, développé avec **Angular**, permet aux utilisateurs de consulter les tableaux de bord interactifs. Les indicateurs peuvent être présentés sous différentes formes graphiques et filtrés selon plusieurs critères afin d'adapter les analyses aux besoins des utilisateurs.

La sécurité est assurée par l'intégration de **Keycloak**, qui prend en charge l'authentification et la gestion des rôles. Les accès aux différentes fonctionnalités sont ainsi contrôlés en fonction du profil de chaque utilisateur.

L'architecture globale peut donc être résumée comme suit :

**Oracle → ETL / Talend → Data Warehouse → API REST
/ Spring Boot → Angular → Tableaux de bord**

Keycloak intervient de manière transversale pour assurer l'authentification et la gestion des autorisations.

Cette organisation permet de séparer clairement les responsabilités des différents composants. Elle facilite ainsi la maintenance de la solution et favorise son évolution, tout en assurant un niveau de sécurité adapté aux besoins de la plateforme décisionnelle.

Architecture globale de la plateforme décisionnelle RH & Crédit

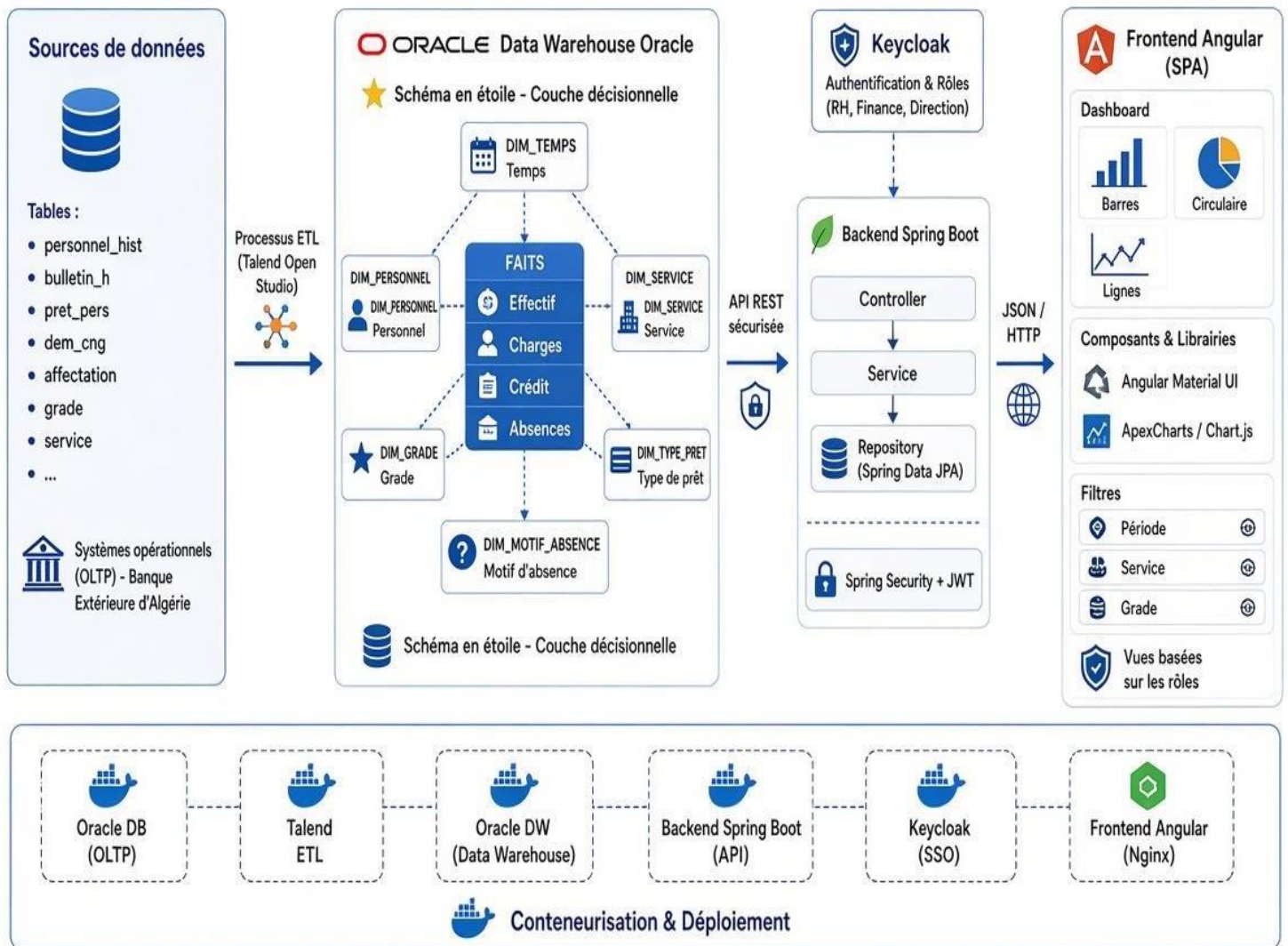


Figure 7 : Architecture globale de la plateforme décisionnelle RH et Crédit

5.2 Conception de l'API Backend (Spring Boot)

L'API Backend a été développée avec **Spring Boot** selon une architecture en couches organisée autour des composants **Controller, Service, Repository et DTO**. Cette organisation permet de séparer clairement les responsabilités de chaque couche et facilite ainsi la maintenance et l'évolution de l'application.

L'accès au **Data Warehouse Oracle**, structuré selon un **schéma en étoile**, est assuré par **Spring Data JPA**. Cette technologie simplifie la gestion des données et facilite les opérations de lecture nécessaires aux traitements analytiques. Elle permet également de maintenir une séparation claire entre la logique métier et l'accès aux données.

La sécurité de l'API repose sur **Spring Security**, intégré à **Keycloak**. L'authentification s'appuie sur des **jetons JWT**, tandis que les rôles et les autorisations permettent de contrôler l'accès aux différentes ressources selon le profil de l'utilisateur, notamment **RH, Finance et Direction**.

L'API REST expose plusieurs **endpoints analytiques sécurisés** dédiés aux principaux indicateurs RH, tels que les effectifs, les charges du personnel et les absences. Elle fournit également les données relatives aux crédits et aux prêts accordés au personnel. Des **filtres dynamiques**, notamment par période, service et grade, permettent d'adapter les résultats aux besoins d'analyse des utilisateurs.

Des **DTO (Data Transfer Objects)** sont utilisés pour structurer les données échangées entre le backend et le frontend. Ils permettent de transmettre uniquement les informations nécessaires aux différentes visualisations et d'adapter le format des réponses aux besoins de l'application Angular.

Par ailleurs, l'API est accompagnée d'une **documentation interactive avec Swagger/OpenAPI**. Celle-ci facilite la consultation et le test des différents endpoints. Une **gestion centralisée des exceptions** a également été mise en place afin d'assurer un traitement cohérent des erreurs et de retourner des réponses adaptées aux clients.

Enfin, l'application backend est **conteneurisée avec Docker et Docker Compose**. Cette approche facilite son déploiement et son intégration avec les autres composants de la plateforme.

Cette conception assure ainsi un **découplage clair entre le backend et le frontend Angular**. Elle contribue également à améliorer la sécurité, la maintenabilité et l'évolutivité de l'application, tout en fournissant un accès structuré aux données décisionnelles.

Architecture de l'API Spring Boot – Couches Controller / Service / Repository

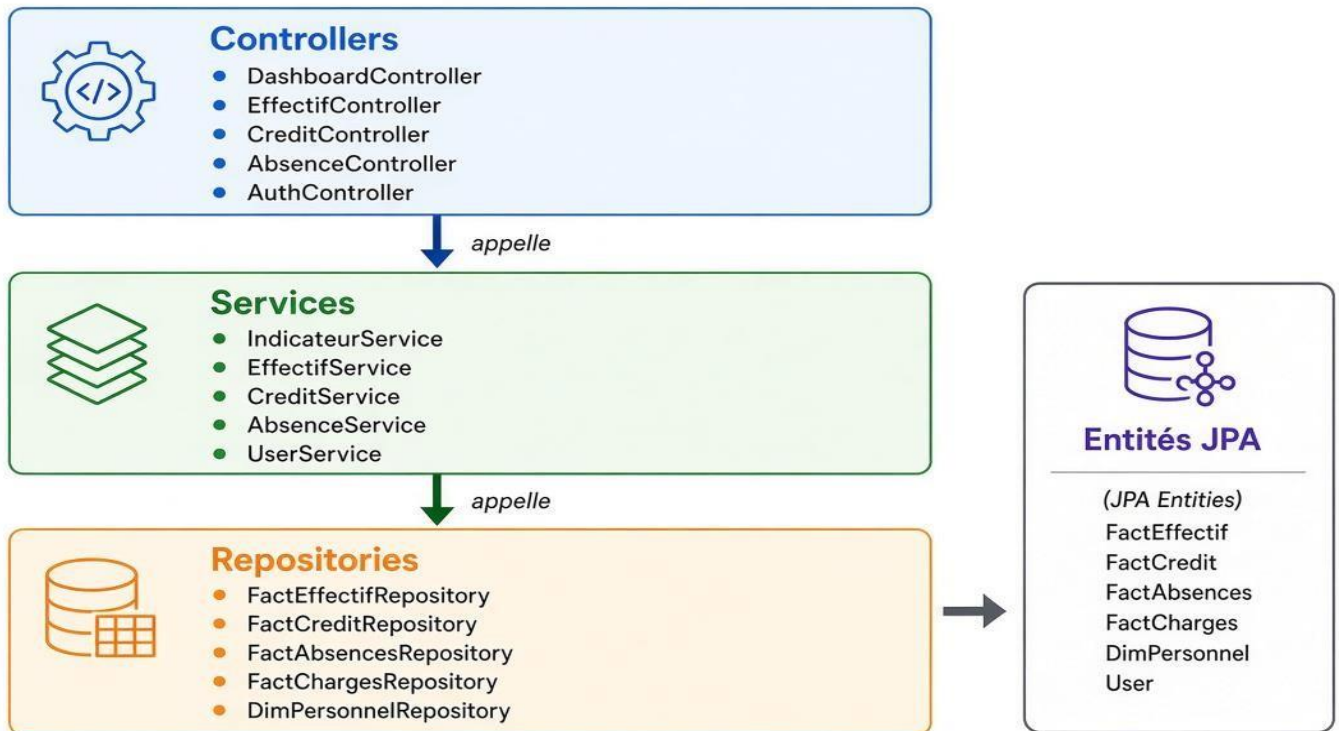


Figure 8: Architecture de l'API Spring Boot

5.3 Modélisation des données (Diagramme de classes)

La **modélisation des données** repose sur le schéma en étoile du **Data Warehouse Oracle** et est transposée dans l'application sous la forme d'un **diagramme de classes Java** basé sur les entités **JPA**.

Le modèle est organisé autour de plusieurs **dimensions**, notamment **Personnel**, issu de la table *personnelle_hist*, ainsi que **Service**, **Grade**, **TypeAdministratifTechnique**, **TypeBulletin**, **TypePret**, **LignePret**, **MotifAbsence** et **DimTemps**. Ces dimensions permettent de caractériser les données et de définir les différents axes d'analyse.

Les **faits** sont représentés par les entités **Affectation**, **Bulletin**, **ParametreFixe**, **PossedeVH**, **PretPersonnel**, **DemandeConge** et **SoldeConge**. Ils regroupent les données nécessaires au calcul et à l'analyse des indicateurs décisionnels.

Les principales associations permettent notamment de relier un **Personnel** à plusieurs affectations, bulletins, prêts et données relatives aux absences. Chaque fait référence aux dimensions nécessaires à son analyse, telles que **Service**, **Grade**, **TypePret** ou **MotifAbsence**, ainsi qu'à la **dimension Temps** pour les analyses périodiques.

Cette modélisation est implémentée avec les annotations **JPA**, notamment **@Entity**, **@ManyToOne** et **@OneToMany**. Elle permet d'assurer la correspondance entre le modèle décisionnel du Data Warehouse et la couche d'accès aux données de l'API Spring Boot.

Enfin, l'utilisation de **DTO** permet de contrôler et de structurer les données exposées par l'API vers le frontend Angular. Cette organisation maintient ainsi une séparation claire entre le modèle de données interne et les informations destinées à la visualisation analytique.

Diagramme de classes – Modèle de données du Data Warehouse RH & Crédit

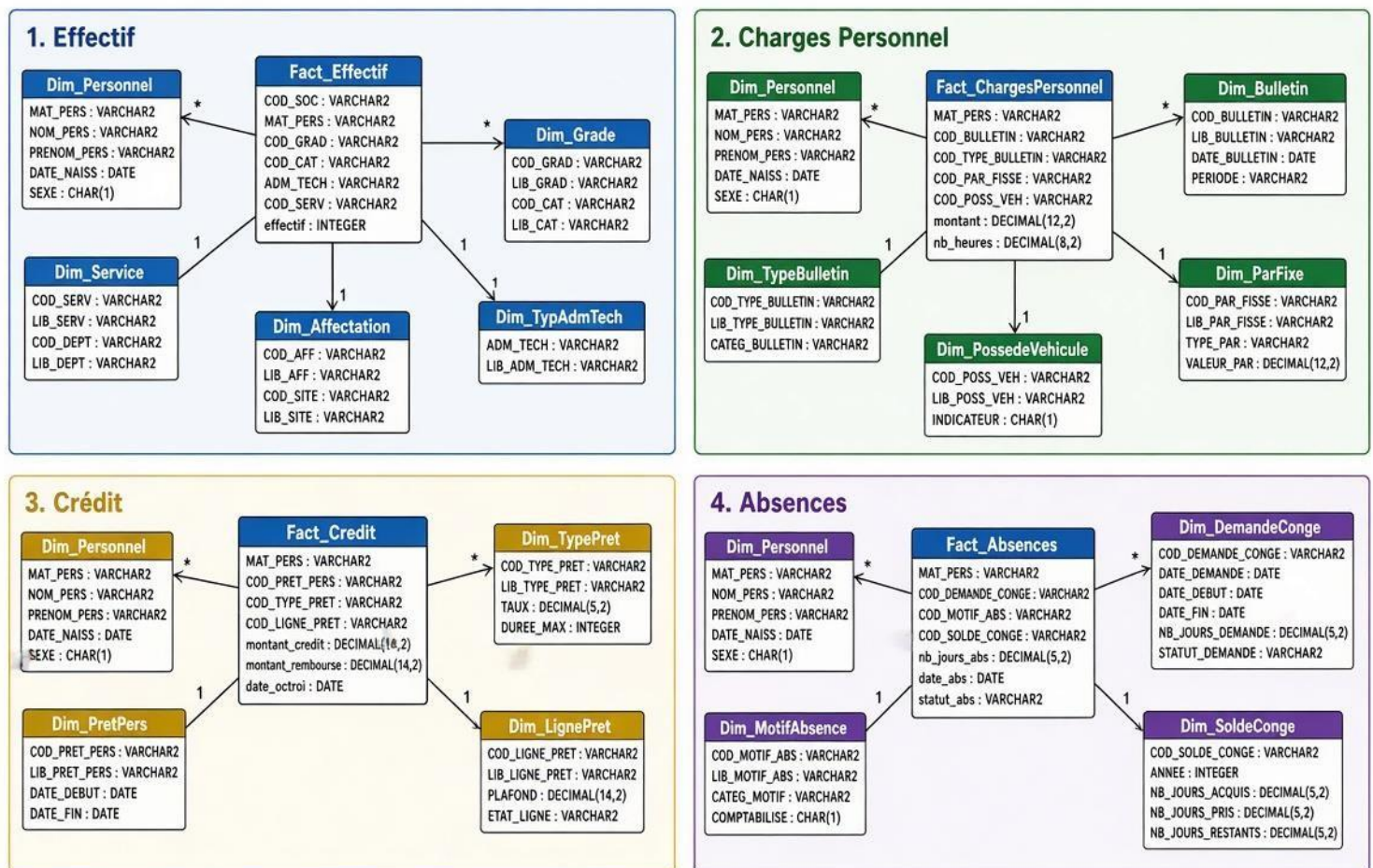


Figure 9 : Diagramme de classes – Modèle de données du Data Warehouse RH et Crédit

5.4 Conception de l'interface utilisateur (Angular)

La conception de l'**interface utilisateur** a été réalisée avec le framework **Angular**, en s'appuyant sur **Angular Material** afin de proposer une interface cohérente, moderne et ergonomique. L'objectif est de faciliter l'accès aux informations décisionnelles et de permettre une navigation simple entre les différentes fonctionnalités de la plateforme.

Pour assurer la **visualisation des indicateurs**, des bibliothèques graphiques telles que **ApexCharts** ont été intégrées à l'application. Elles permettent de représenter les données issues du Data Warehouse sous différentes formes, notamment à travers des graphiques, des tableaux et des indicateurs synthétiques. Ces visualisations facilitent ainsi la compréhension et l'interprétation des données RH et Crédit.

L'application intègre également une **authentification centralisée avec Keycloak**. Les rôles **RH, Finance et Direction** permettent de contrôler l'accès aux différentes fonctionnalités et aux informations disponibles selon le profil de l'utilisateur.

Des **filtres dynamiques** sont proposés afin d'affiner les analyses selon plusieurs critères, notamment la période, le service et le grade. Les résultats affichés sont ainsi adaptés aux besoins spécifiques de chaque utilisateur.

Enfin, les différentes interfaces ont été conçues pour assurer une **navigation claire et intuitive**. L'utilisation combinée de graphiques, de tableaux et de KPI permet aux responsables RH et financiers de consulter rapidement les informations importantes et de suivre l'évolution des principaux indicateurs de la plateforme.

5.5 Architecture de sécurité et gestion des rôles (Keycloak)

L'**architecture de sécurité** de la plateforme repose sur l'intégration de **Keycloak** comme serveur central de gestion des identités et des accès. Il est associé à **Spring Security** au niveau du backend et à **Angular** au niveau du frontend. Cette organisation permet de séparer clairement les responsabilités liées à l'authentification et au contrôle des accès.

Keycloak assure de manière centralisée l'**authentification des utilisateurs**, notamment la connexion, la déconnexion et la gestion des jetons d'accès. Il prend également en charge les rôles métier définis dans le projet, tels que **RH, Finance et Direction**.

Lorsqu'un utilisateur s'authentifie, Keycloak génère un **jeton JWT** contenant les informations nécessaires à son identification ainsi que les rôles qui lui sont attribués. Ce jeton est ensuite transmis à l'API **Spring Boot**, configurée comme serveur de ressources **OAuth2**. À chaque requête vers une ressource protégée, le backend vérifie la validité du jeton et les autorisations associées à l'utilisateur.

Des contrôles supplémentaires sont appliqués au niveau des contrôleurs et des services grâce aux mécanismes de sécurité de **Spring Security**, notamment l'annotation **@PreAuthorize**. Celle-ci permet de restreindre l'accès à certaines ressources selon le rôle de l'utilisateur. Par exemple, les données sensibles relatives aux charges du personnel peuvent être accessibles uniquement aux profils **RH** et **Direction**.

Du côté du frontend, **Angular** participe à la gestion des jetons et adapte l'interface selon le rôle de l'utilisateur connecté. Les menus, les tableaux de bord et certaines fonctionnalités peuvent ainsi être affichés ou masqués en fonction des autorisations accordées.

Cette architecture centralisée permet donc de contrôler précisément les accès aux différentes ressources de la plateforme. Elle renforce la confidentialité des données, facilite la gestion des rôles et assure une séparation claire entre l'authentification, l'autorisation et la logique métier. Elle répond ainsi aux exigences de sécurité liées à une plateforme décisionnelle manipulant des données RH et financières.

5.6 Conclusion

L'architecture définie dans ce chapitre permet de proposer une solution claire, modulaire et sécurisée, répondant aux objectifs de la plateforme décisionnelle **RH & Crédit**. La solution repose sur une architecture en couches, organisée autour d'un **Data Warehouse Oracle structuré selon un schéma en étoile**, d'une **API REST développée avec Spring Boot** et d'une **interface utilisateur conçue avec Angular**.

Cette organisation permet de séparer clairement les responsabilités liées au stockage, au traitement et à la présentation des données. La modélisation réalisée à travers un **diagramme de classes** assure également une bonne correspondance entre le modèle décisionnel et la couche d'accès aux données de l'application.

En complément, l'intégration de **Keycloak** permet de centraliser l'authentification des utilisateurs et la gestion des rôles métier. Elle contribue ainsi à renforcer la sécurité de la plateforme et à contrôler l'accès aux différentes fonctionnalités selon le profil de chaque utilisateur.

Les choix architecturaux retenus constituent donc une base technique adaptée à la réalisation d'une application **robuste, évolutive et sécurisée**, destinée aux responsables RH et financiers de la **Banque Extérieure d'Algérie (BEA)**.

Le chapitre suivant sera consacré à la **réalisation et à l'implémentation de la solution**. Il présentera les différentes étapes de développement ainsi que les interfaces et fonctionnalités mises en œuvre.

Chapitre 6 : Réalisation et implémentation

6.1 Environnement de développement et outils utilisés

L'**environnement de développement** retenu pour la réalisation de la plateforme décisionnelle **RH & Crédit** regroupe plusieurs technologies et outils adaptés aux besoins fonctionnels et techniques du projet.

La partie **backend** a été développée en **Java** avec le framework **Spring Boot**, en utilisant **Spring Data JPA** pour assurer l'accès et la gestion des données. La sécurité repose sur **Spring Security**, associé à **Keycloak** pour centraliser l'authentification et la gestion des rôles métier, notamment **RH, Finance et Direction**. Le backend expose également une **API REST** permettant d'assurer la communication avec l'application frontend. La documentation et le test des différents services sont facilités par l'utilisation de **Swagger/OpenAPI**.

La partie **frontend** a été développée avec **Angular**. Le framework **Angular Material** est utilisé pour concevoir une interface claire, cohérente et adaptée aux besoins des utilisateurs. La bibliothèque **ApexCharts** permet de représenter les indicateurs sous différentes formes graphiques et de proposer des visualisations interactives facilitant l'analyse des données.

Les données sources ainsi que le **Data Warehouse** sont gérés avec **Oracle**. Ce dernier constitue le support de stockage du modèle décisionnel, organisé selon un **schéma en étoile**. Les opérations d'extraction, de transformation et de chargement (**ETL**) sont réalisées avec **Talend Open Studio**. Par ailleurs, **PL/SQL** est utilisé pour certaines opérations sur la base de données ainsi que pour la préparation de requêtes nécessaires aux traitements analytiques.

La gestion et le versionnement du code source sont assurés avec **Git** et **GitLab**. Ces outils permettent de suivre les différentes versions du projet et de faciliter la gestion du développement. Enfin, la solution est **conteneurisée avec Docker**, tandis que **Docker Compose** permet d'orchestrer les différents services nécessaires à son fonctionnement.

Cet environnement technologique fournit ainsi une base cohérente pour l'ensemble du projet. Il couvre les différentes étapes de la solution, depuis la préparation et l'intégration des données jusqu'au développement de l'API, à la sécurisation des accès et à la visualisation des indicateurs décisionnels.

6.2 Mise en œuvre du Data Warehouse et du processus ETL

La mise en œuvre du **Data Warehouse** a porté sur sa conception et son déploiement sous **Oracle**, selon une organisation en **schéma en étoile** adaptée aux besoins d'analyse des domaines RH et Crédit. Les différentes tables de dimensions ont été définies afin de regrouper les informations nécessaires aux différents axes d'analyse. Elles comprennent notamment **Personnel**, alimentée à partir de la table *personnelle_hist*, ainsi que **Service**, **Grade**, **TypeAdministratifTechnique**, **TypeBulletin**, **TypePret**, **LignePret**, **MotifAbsence** et la **dimension Temps**.

Ces dimensions permettent de caractériser les données et de faciliter leur analyse selon différents critères. Elles sont associées aux tables de faits qui regroupent les informations nécessaires au calcul et au suivi des indicateurs décisionnels.

Les **tables de faits** mises en œuvre concernent notamment **Affectation**, **Bulletin**, **ParametreFixe**, **PossedeVH**, **PretPersonnel**, **DemandeConge** et **SoldeConge**. Elles permettent de centraliser les données relatives aux effectifs, aux charges du personnel, aux crédits et aux absences dans une structure adaptée aux traitements analytiques.

L'alimentation du Data Warehouse est assurée par un processus **ETL développé avec Talend Open Studio**. Les données sont extraites des différentes tables opérationnelles Oracle, puis transformées avant leur chargement dans les tables du modèle décisionnel. Les traitements réalisés comprennent notamment le nettoyage des données, l'harmonisation des formats, la normalisation ainsi que leur adaptation à la structure des dimensions et des faits. Des contrôles permettent également de vérifier les données traitées et d'identifier les éventuels rejets.

En complément, des procédures et requêtes **PL/SQL** ont été utilisées au niveau de l'entrepôt pour réaliser certains traitements et agrégations nécessaires aux analyses. Les données ainsi préparées sont ensuite exploitées par l'**API backend** afin d'alimenter les différents indicateurs et tableaux de bord de la plateforme.

Cette mise en œuvre permet ainsi de disposer d'un **Data Warehouse structuré, cohérent et adapté aux besoins analytiques** du projet. Il constitue la base de la restitution des indicateurs RH et Crédit à travers l'application décisionnelle.

6.3 Développement de l'API REST (Spring Boot, Spring Data JPA)

Le développement de l'API REST a été réalisé avec **Spring Boot** et **Spring Data JPA** afin de fournir un accès structuré et sécurisé aux données analytiques du **Data Warehouse Oracle**. L'API expose plusieurs endpoints REST permettant de récupérer les principaux indicateurs RH, notamment les effectifs, les charges du personnel et les absences, ainsi que les indicateurs relatifs au Crédit. Elle prend également en charge des **filtres dynamiques**, tels que la période, le service et le grade, afin d'adapter les résultats aux besoins de l'application Angular.

L'utilisation de **Spring Data JPA** permet d'assurer la correspondance entre les entités Java et les tables de faits et de dimensions du schéma en étoile. Cette approche facilite l'accès aux données et la réalisation des traitements nécessaires aux analyses. Lorsque cela est nécessaire, des **requêtes natives Oracle** peuvent être utilisées pour réaliser certains traitements spécifiques ou optimiser l'exploitation des données.

L'API repose sur une **architecture en couches**. La couche *Repository* assure l'accès aux données, tandis que la couche *Service* centralise la logique métier et les traitements d'agrégation. Les *Controllers* gèrent les requêtes HTTP et transmettent les résultats au frontend. Des **DTO (Data Transfer Objects)** sont utilisés pour structurer les données retournées et adapter les réponses aux besoins des tableaux de bord.

La **sécurité de l'API** est assurée par **Spring Security**, intégré à **Keycloak**. Cette organisation permet de gérer l'authentification à l'aide de **jetons JWT** et de contrôler l'accès aux ressources selon les rôles attribués aux utilisateurs. Les profils **RH, Finance et Direction** disposent ainsi de droits adaptés à leurs responsabilités.

Enfin, l'API est documentée avec **Swagger/OpenAPI**, ce qui facilite la consultation et le test des endpoints ainsi que la maintenance du projet. Son intégration dans un environnement **Docker** simplifie également son déploiement et son fonctionnement avec les autres composants de la plateforme.

Ainsi, les choix réalisés permettent de disposer d'une API **structurée, sécurisée et maintenable**, assurant la communication entre le Data Warehouse Oracle et l'interface Angular de la plateforme décisionnelle RH & Crédit.

6.4 Développement du tableau de bord (Angular, ApexCharts/Chart.js)

Le développement du **tableau de bord décisionnel** a été réalisé avec le framework **Angular**, en s'appuyant sur **Angular Material** afin de concevoir une interface moderne, responsive et ergonomique. La bibliothèque **ApexCharts** a également été intégrée pour représenter de manière interactive les différents indicateurs RH et Crédit issus du Data Warehouse. Ces visualisations permettent de présenter les informations sous une forme synthétique et facilitent leur interprétation par les utilisateurs.

L'application intègre un mécanisme d'**authentification sécurisé avec Keycloak**, permettant de gérer les rôles des utilisateurs, notamment **RH, Finance et Direction**, et de contrôler leur accès aux différentes fonctionnalités de la plateforme. Des **filtres dynamiques**, portant notamment sur la période, le service et le grade, permettent d'affiner les analyses selon les critères sélectionnés. Les données affichées sont ainsi actualisées en fonction des paramètres choisis.

L'architecture du frontend repose sur une organisation **modulaire des composants Angular**. Elle comprend notamment des composants dédiés aux filtres, aux cartes d'indicateurs, aux graphiques et aux tableaux de données. Cette organisation favorise la **réutilisabilité, la maintenabilité et l'évolution** de l'application. Les échanges avec le backend sont réalisés à travers l'**API REST Spring Boot**, qui fournit les données nécessaires à l'**alimentation** des différentes visualisations, notamment celles relatives aux effectifs, aux charges du personnel, aux absences et aux crédits.

Les indicateurs sont représentés à l'aide de plusieurs types de visualisations, tels que les **diagrammes en barres, les courbes et les diagrammes circulaires**, selon la nature des données à analyser. Des fonctionnalités interactives, notamment les **info-bulles (tooltips)**, permettent d'obtenir des informations détaillées lors de la consultation des graphiques. Des fonctionnalités d'export peuvent également être proposées pour faciliter la réutilisation des données et des résultats présentés.

Ainsi, le tableau de bord offre une **interface claire, interactive et adaptée aux besoins des utilisateurs métier**. Il permet aux responsables RH et financiers de consulter les principaux indicateurs, d'affiner leurs analyses et de disposer d'informations synthétiques pour accompagner la prise de décision.

6.5 Mise en place de l'authentification et de la gestion des rôles (Keycloak)

La mise en œuvre du mécanisme d'**authentification et de gestion des rôles** repose sur **Keycloak**, une solution open source d'**Identity and Access Management (IAM)** intégrée à l'architecture de la plateforme. Keycloak est associé à **Spring Security** au niveau du backend et à **Angular** au niveau du frontend. Cette organisation permet de centraliser la gestion des utilisateurs, des rôles et des autorisations tout en assurant une authentification sécurisée.

Après une authentification réussie, **Keycloak délivre un jeton JWT** contenant les informations nécessaires à l'identification de l'utilisateur ainsi que les rôles qui lui sont attribués. Ce jeton est ensuite transmis par l'application Angular lors des requêtes adressées à l'API afin de permettre au backend de vérifier l'identité et les autorisations de l'utilisateur.

Au niveau du **backend**, l'API développée avec **Spring Boot** est configurée comme un **Resource Server OAuth2**. Elle vérifie la validité des jetons reçus avant d'autoriser l'accès aux ressources protégées. **Spring Security** applique ensuite les règles d'autorisation définies selon les rôles attribués, notamment **RH, Finance et Direction**. Cette organisation permet de limiter l'accès aux données et aux fonctionnalités correspondant aux responsabilités de chaque profil.

Du côté du **frontend**, l'intégration de Keycloak avec **Angular** permet de gérer l'authentification et de protéger les différentes routes de l'application. Elle permet également de prendre en charge le cycle de vie des jetons, notamment leur expiration et leur renouvellement. Cette intégration peut aussi s'appuyer sur le principe du **Single Sign-On (SSO)**, permettant aux utilisateurs authentifiés d'accéder aux ressources autorisées sans devoir s'authentifier à nouveau auprès de chaque service.

Ainsi, l'architecture mise en place renforce la **sécurité globale de la plateforme décisionnelle** tout en séparant clairement l'authentification, l'autorisation et la logique métier. La centralisation des mécanismes de sécurité facilite également la gestion des droits d'accès et permet de faire évoluer les rôles selon les besoins futurs de la plateforme.

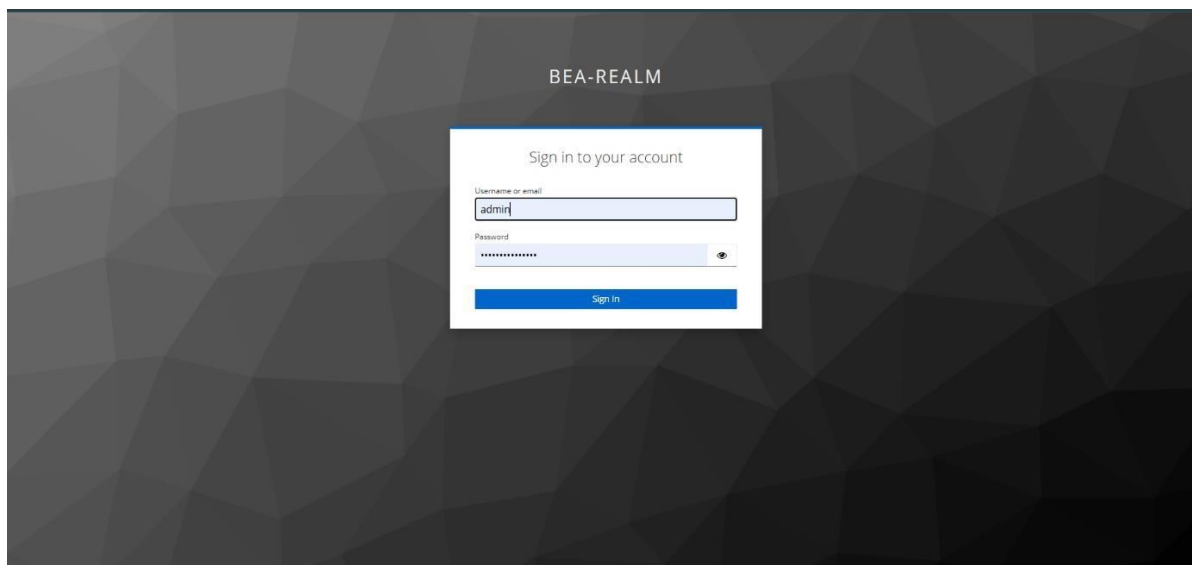


Figure 10 : Interface d'authentification

6.6 Conteneurisation et déploiement avec Docker et Docker Compose

La **conteneurisation et le déploiement** de la plateforme ont été réalisés avec **Docker** et **Docker Compose** afin de fournir un environnement d'exécution isolé, reproductible et portable. Chaque composant principal de la solution est exécuté dans un conteneur dédié, notamment l'**API Spring Boot**, l'**application frontend Angular** et le serveur d'authentification **Keycloak**. La base de données **Oracle**, qui héberge le Data Warehouse, peut également être intégrée à cet environnement conteneurisé selon les besoins du déploiement.

Les différentes images Docker sont construites à partir de **Dockerfiles**. Ces fichiers permettent de définir les dépendances, la configuration et les paramètres nécessaires à l'exécution de chaque composant. Cette organisation garantit une configuration homogène et facilite la reproduction de l'environnement sur différentes machines.

L'orchestration des services est assurée par **Docker Compose**. Il permet de regrouper les différents conteneurs au sein d'une configuration commune et de gérer leurs interactions. Il facilite notamment la configuration du réseau, des variables d'environnement, des volumes persistants et des ports utilisés par les services. Le démarrage et l'arrêt de la plateforme peuvent ainsi être réalisés de manière centralisée.

La conteneurisation facilite également la gestion des différents environnements de développement et de test. Elle limite les problèmes liés aux différences de configuration entre les machines et simplifie le déploiement des différents composants de la solution.

Ainsi, l'utilisation de **Docker et Docker Compose** contribue à rendre la plateforme plus **portable, maintenable et facilement déployable**. Cette approche constitue également une base adaptée à l'évolution future de la solution et à l'intégration de nouveaux services.

6.7 Présentation des interfaces réalisées

La présentation des **interfaces développées** met en évidence une application web réalisée avec **Angular**, conçue pour offrir une expérience utilisateur moderne, ergonomique et adaptée aux besoins des responsables RH et financiers. L'application est organisée autour de plusieurs écrans permettant de consulter et d'analyser les principaux indicateurs de la plateforme décisionnelle.

L'**interface d'authentification**, sécurisée par **Keycloak**, permet aux utilisateurs de se connecter tout en assurant une gestion des accès selon leur rôle, notamment **RH, Finance ou Direction**. Après une authentification réussie, l'utilisateur accède aux fonctionnalités correspondant à son profil et est orienté vers le tableau de bord approprié.

Le **tableau de bord principal** présente une vue synthétique des principaux indicateurs, tels que l'effectif global, les charges du personnel, le volume des crédits et les absences. Ces informations sont représentées à l'aide de **cartes KPI, de tableaux et de graphiques interactifs développés avec ApexCharts**, facilitant ainsi leur lecture et leur interprétation.

Des interfaces spécifiques permettent également d'effectuer une analyse plus détaillée des différents domaines. L'écran **Effectif** présente la répartition du personnel selon plusieurs critères, notamment le service et le grade, avec la possibilité d'appliquer des filtres dynamiques. L'écran **Charges du personnel** permet de consulter les informations relatives aux bulletins et aux paramètres fixes. L'écran **Crédit** permet d'analyser les prêts selon leur type et leurs différentes caractéristiques. Enfin, l'écran **Absences** permet de suivre les demandes de congés, les motifs d'absence et les soldes disponibles.

L'ensemble des interfaces repose sur une **navigation structurée et intuitive**, accompagnée d'un menu latéral facilitant l'accès aux différentes fonctionnalités. Les filtres disponibles, notamment ceux liés à la période, au service et au grade, permettent d'adapter les résultats affichés aux besoins d'analyse.

Cette organisation offre ainsi une interface **claire, interactive et adaptée aux utilisateurs métier**. Elle facilite l'exploitation des données RH et Crédit et permet aux responsables concernés de disposer rapidement d'informations synthétiques pour accompagner la prise de décisio

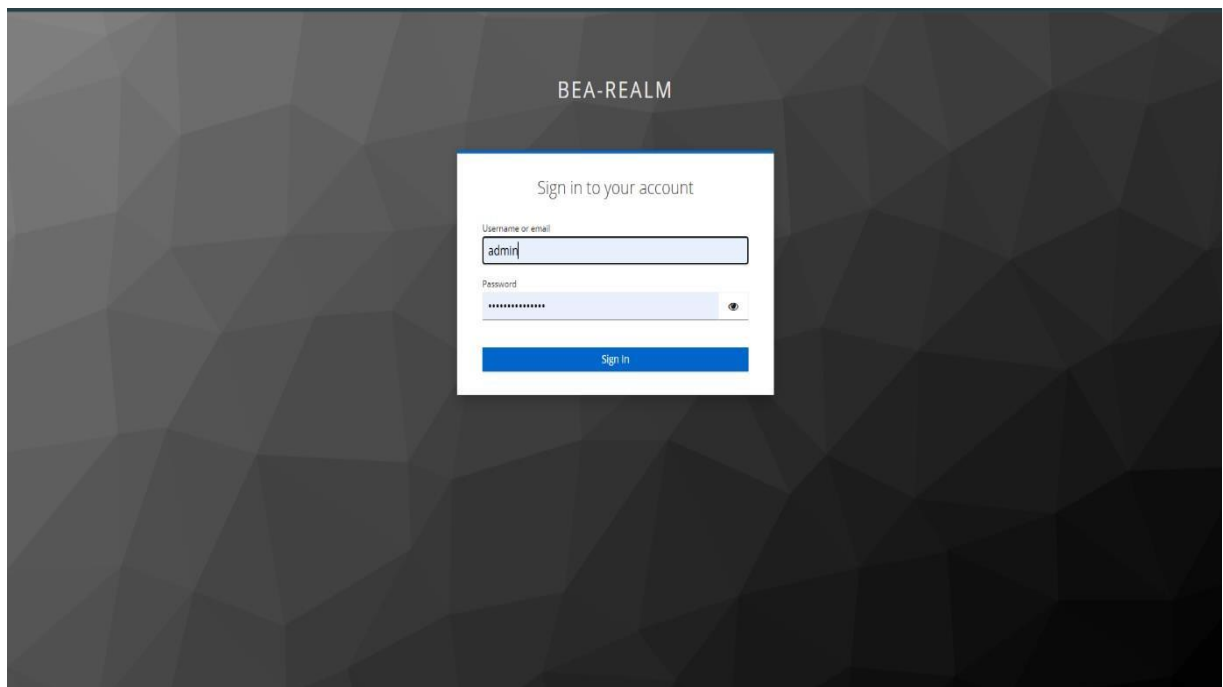


Figure 10 : Interface d'authentification

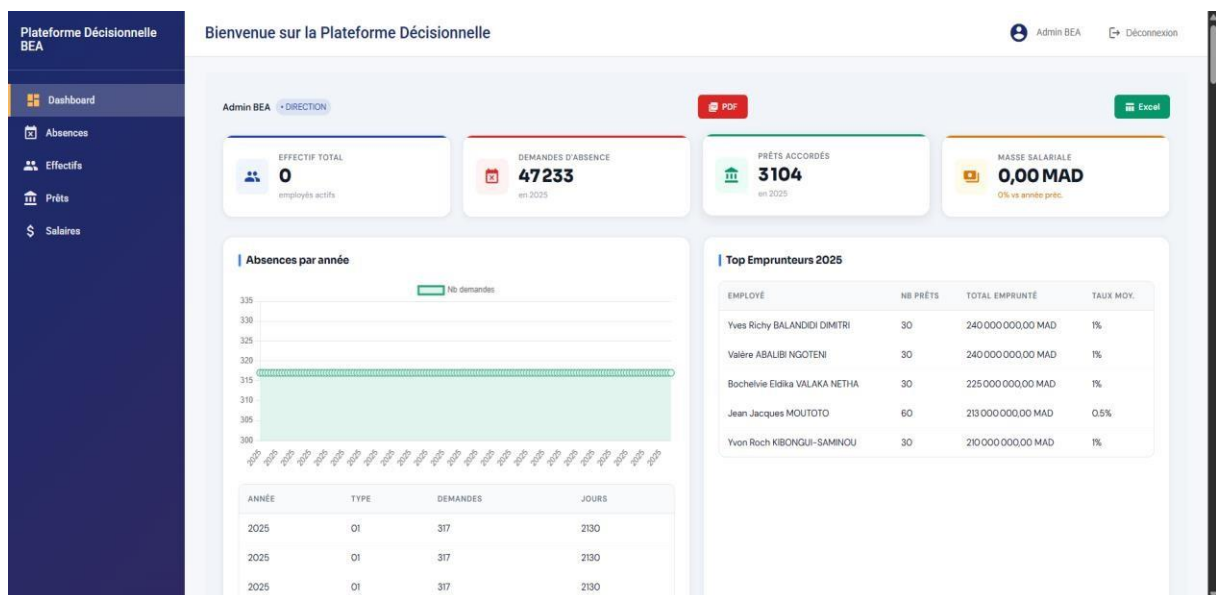


Figure 11 : Tableau de bord principal

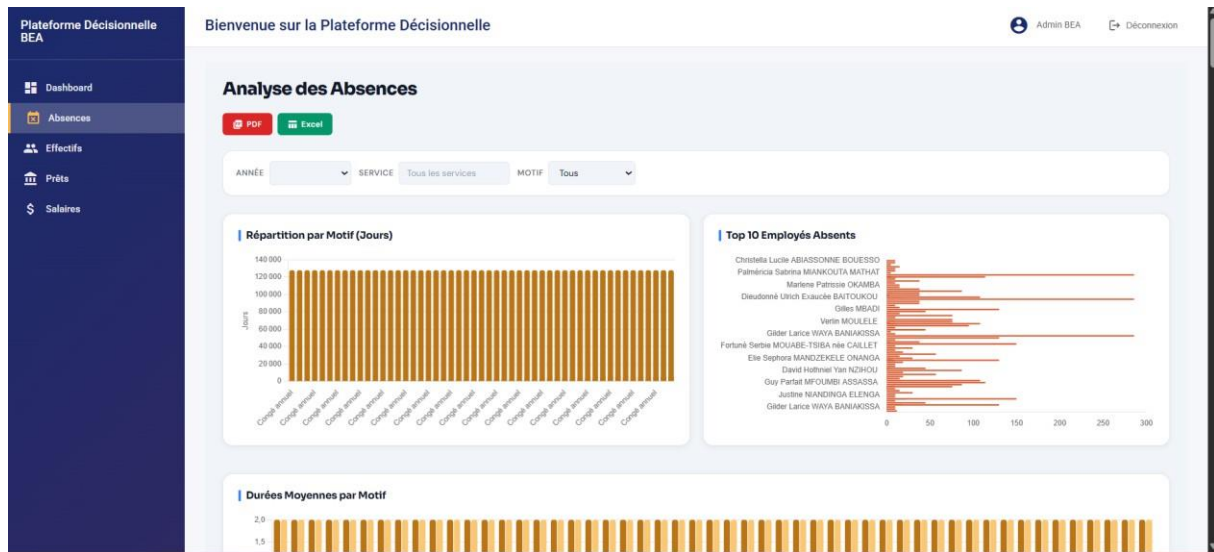


Figure 12 : Dashboard Absences

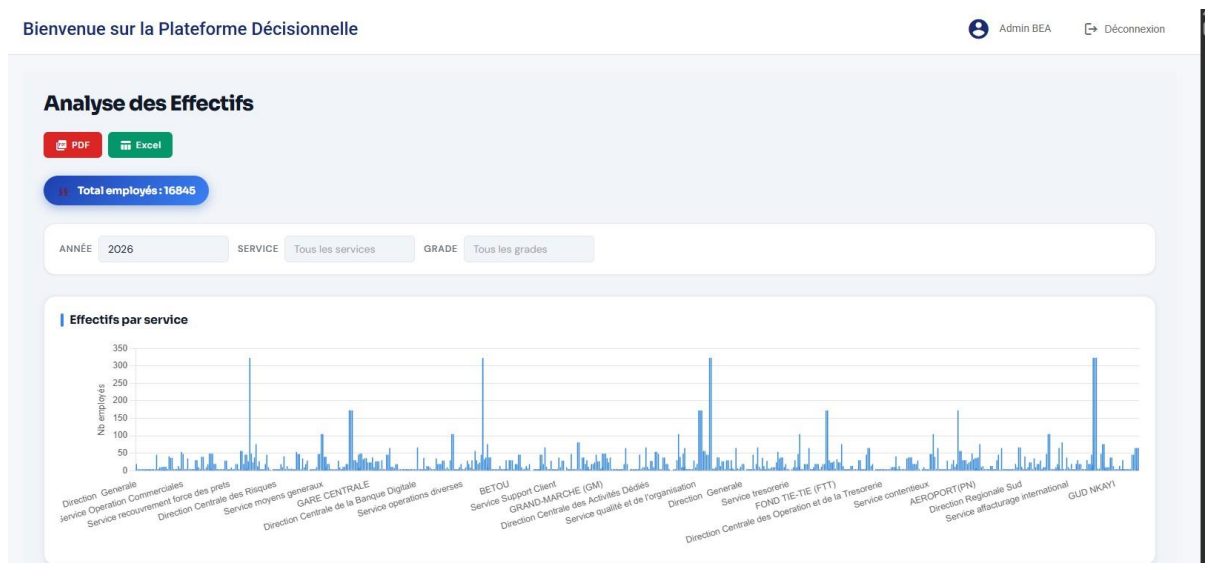


Figure 13 : Dashboard Effectifs

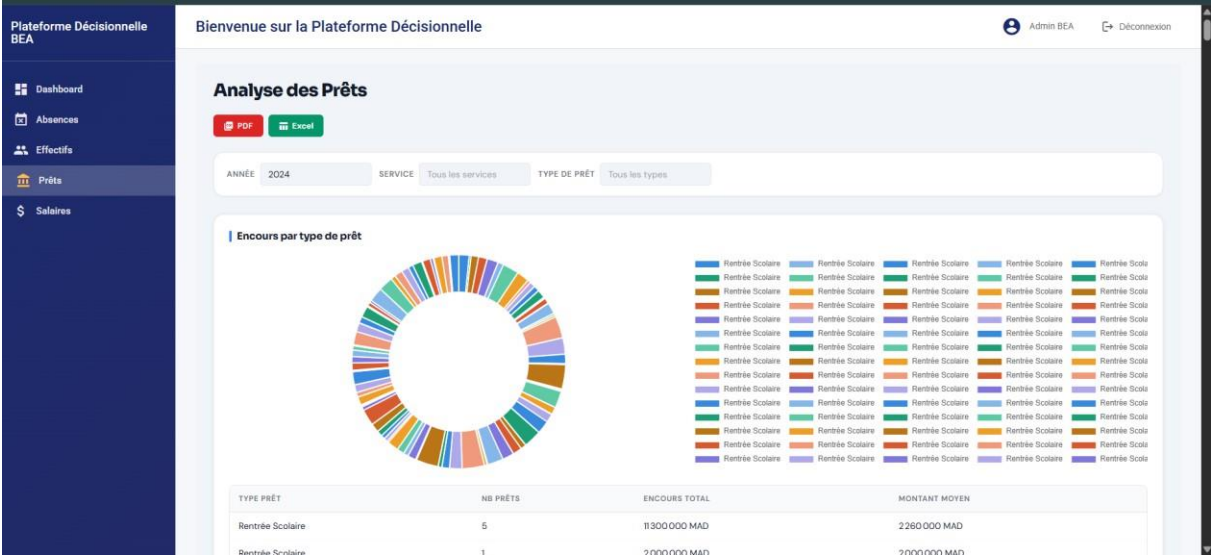


Figure14 : Dashboard Prêts

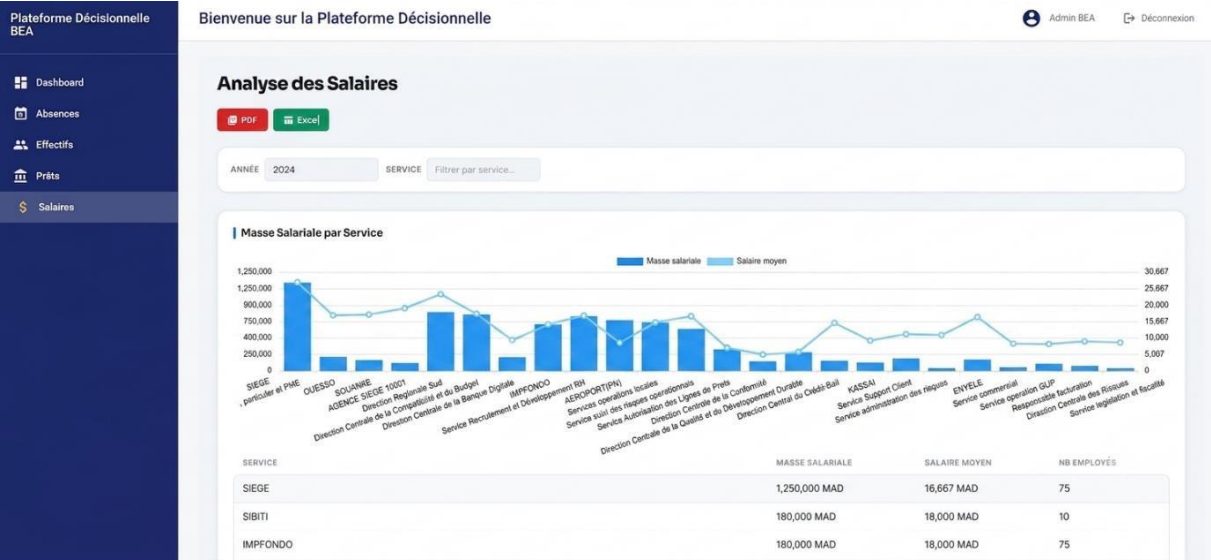


Figure 15 : Dashboard Salaires

6.8 Conclusion

Ce chapitre a permis de présenter les principales étapes de **réalisation et d'implémentation** de la plateforme décisionnelle **RH & Crédit**. Dans un premier temps, l'environnement de développement ainsi que les différents outils et technologies utilisés ont été présentés. La mise en œuvre du **Data Warehouse Oracle**, reposant sur un schéma en étoile, ainsi que le processus **ETL** destiné à l'extraction, à la transformation et au chargement des données ont ensuite été détaillés.

La partie applicative a ensuite porté sur le développement de l'**API REST** avec **Spring Boot** et **Spring Data JPA**. Cette API assure l'accès aux données analytiques et leur mise à disposition pour l'application frontend. Le développement du tableau de bord avec **Angular**, associé à la bibliothèque **ApexCharts**, a également été présenté afin d'illustrer les mécanismes de visualisation et d'analyse des principaux indicateurs RH et Crédit.

La **sécurisation de la plateforme** a également été abordée à travers l'intégration de **Keycloak**, permettant de centraliser l'authentification et la gestion des rôles et des droits d'accès. La solution a ensuite été conteneurisée avec **Docker** et **Docker Compose**, facilitant son déploiement et assurant un environnement cohérent et portable.

Enfin, la présentation des différentes **interfaces développées** a permis d'illustrer concrètement les fonctionnalités de la plateforme, notamment la consultation des indicateurs, l'utilisation des filtres dynamiques et l'analyse des données à travers différents types de visualisations.

L'ensemble de ces réalisations a permis d'aboutir à une plateforme décisionnelle **structurée, sécurisée et fonctionnelle**, capable de centraliser, d'analyser et de visualiser les données relatives aux RH et au Crédit. La solution répond ainsi aux principaux objectifs définis au début du projet et constitue un outil d'aide à la décision adapté aux besoins des responsables **RH, Finance et Direction**.

Chapitre 7 : Tests et validation

7.1 Stratégie de tests

La stratégie de **tests** adoptée pour la plateforme décisionnelle **RH & Crédit** repose sur une approche à plusieurs niveaux. Elle vise à vérifier la fiabilité, la sécurité, les performances et la conformité fonctionnelle de la solution. Les vérifications concernent principalement le **Data Warehouse**, le processus **ETL**, l'API backend et l'application web.

Au niveau **unitaire**, des tests peuvent être réalisés sur les principaux composants du backend développé avec **Spring Boot**, notamment les services métier, les repositories JPA et les contrôleurs REST, à l'aide de **JUnit** et **Mockito**. Du côté frontend, les composants et services Angular peuvent également être testés avec **Jasmine** et **Karma**. Ces tests permettent de vérifier le comportement de chaque unité logicielle à partir de différents scénarios, notamment avec des données valides, des cas limites ou des situations générant des erreurs.

Les **tests d'intégration** permettent ensuite de vérifier les interactions entre les différentes couches de la plateforme. Une attention particulière est accordée aux requêtes analytiques, aux traitements **PL/SQL** et aux transformations réalisées avec **Talend Open Studio**. L'objectif est notamment de vérifier que les données provenant des sources Oracle sont correctement extraites, transformées puis chargées dans les tables du Data Warehouse. La communication entre l'API Spring Boot et la base Oracle est également vérifiée afin de s'assurer du bon fonctionnement de la couche d'accès aux données.

La sécurité constitue également un volet important des tests. L'intégration de **Keycloak avec Spring Security** est vérifiée afin de contrôler le processus d'authentification ainsi que les droits associés aux différents rôles. Les tests portent notamment sur la validité des **jetons JWT**, l'accès aux ressources protégées et la séparation des permissions entre les profils **RH, Finance et Direction**. L'objectif est de s'assurer qu'un utilisateur ne puisse accéder qu'aux fonctionnalités correspondant à ses autorisations.

Des **tests fonctionnels** sont également effectués sur l'application Angular afin de vérifier les principaux parcours utilisateurs. Ils concernent notamment l'authentification, la consultation des tableaux

de bord, l'utilisation des filtres dynamiques et l'affichage des indicateurs RH et Crédit. Les différentes visualisations développées avec **ApexCharts** sont vérifiées afin de s'assurer que les données affichées correspondent aux résultats fournis par l'API.

Des tests de **non-régression** sont réalisés lors des évolutions importantes de la plateforme. Ils permettent de vérifier que l'ajout ou la modification d'une fonctionnalité n'entraîne pas de régression au niveau des fonctionnalités existantes. Des vérifications de performance peuvent également être effectuées sur les principales requêtes analytiques et les endpoints de l'API afin d'évaluer leurs temps de réponse avec des volumes de données représentatifs.

Cette stratégie de tests permet ainsi de couvrir progressivement les principaux aspects fonctionnels et techniques de la plateforme. Elle contribue à identifier les anomalies, à vérifier la cohérence des données et à améliorer la **fiabilité, la sécurité et la qualité globale** de la solution avant sa mise en exploitation.

7.2 Tests unitaires et fonctionnels

Dans le cadre de la validation de la plateforme décisionnelle **RH & Crédit**, une stratégie de tests structurée a été mise en place afin de vérifier la fiabilité, la robustesse et la conformité fonctionnelle de la solution développée. Cette démarche couvre les différentes couches de l'architecture, du **Data Warehouse et du backend jusqu'à l'application frontend**. Elle permet ainsi de vérifier le fonctionnement des composants individuellement ainsi que leurs interactions au sein du système global.

Les **tests unitaires** ont principalement été réalisés au niveau du backend développé avec **Spring Boot**, en utilisant **JUnit 5 et Mockito**. Les principaux services métier, notamment ceux liés à la gestion des effectifs, aux charges du personnel, aux crédits et aux absences, ont été testés de manière isolée. Les dépendances externes, telles que les repositories JPA et les accès au Data Warehouse, ont été simulées afin de vérifier la logique métier indépendamment des autres composants. Ces tests ont permis de contrôler le comportement de l'application dans différents scénarios, notamment les cas limites et la gestion des exceptions.

Du côté du **frontend Angular**, des tests unitaires ont également été réalisés avec **Jasmine et Karma**. Ils permettent de vérifier le comportement des composants et des services utilisés dans l'application, notamment pour la gestion des filtres dynamiques, l'affichage des indicateurs et la gestion des rôles utilisateurs.

Les **tests fonctionnels et d'intégration** ont ensuite permis d'évaluer le comportement de la plateforme dans des conditions proches de son utilisation réelle. Des tests des **API REST** ont été réalisés avec **Postman** et, lorsque nécessaire, avec **RestAssured**, afin de vérifier le fonctionnement des différents endpoints exposés par le backend. Ces tests ont notamment porté sur l'authentification via **Keycloak**, la récupération des indicateurs RH et Crédit ainsi que l'application des filtres selon la période, le service et le grade.

Une attention particulière a également été accordée aux **mécanismes de sécurité**. Les différents profils utilisateurs, notamment **RH, Finance et Direction**, ont été testés afin de vérifier que chaque rôle dispose uniquement des accès correspondant à ses autorisations. La validité des jetons JWT et la protection des ressources sécurisées ont également été vérifiées.

Des **tests de bout en bout (End-to-End)** ont enfin été réalisés afin

de valider les principaux parcours utilisateurs. Ils couvrent notamment la connexion à la plateforme, l'accès aux tableaux de bord selon le rôle de l'utilisateur, la consultation des graphiques interactifs développés avec **ApexCharts** ainsi que l'utilisation des filtres dynamiques. La cohérence entre les données présentes dans le **Data Warehouse Oracle**, celles retournées par l'API et celles affichées dans les tableaux de bord a également été vérifiée.

Cette démarche de validation a permis d'identifier et de corriger certaines anomalies liées à l'intégration entre le **Data Warehouse, le backend et le frontend**. L'ensemble des tests réalisés a ainsi contribué à améliorer la stabilité, la fiabilité et la cohérence de la plateforme, tout en garantissant le bon fonctionnement des principales fonctionnalités décisionnelles destinées aux responsables **RH, Finance et Direction**.

7.3 Tests de performance

Les tests de performance ont été réalisés afin d'évaluer le comportement de la plateforme décisionnelle RH & Crédit dans des conditions représentatives de son utilisation par les responsables des fonctions RH, Finance et Direction. L'évaluation a porté sur les principales composantes de la solution, notamment les requêtes analytiques exécutées sur le Data Warehouse Oracle, les services REST développés avec Spring Boot ainsi que l'interface frontend Angular.

Les scénarios de charge ont consisté à simuler un nombre croissant d'utilisateurs réalisant différentes opérations représentatives de l'utilisation de la plateforme. Ces opérations comprennent notamment l'authentification, la consultation des tableaux de bord, l'application de filtres selon la période, le service ou le grade, ainsi que la récupération des indicateurs relatifs aux effectifs, aux charges du personnel, aux crédits et aux absences.

Les résultats des tests ont permis d'évaluer les temps de réponse des principaux endpoints de l'API ainsi que le temps nécessaire au chargement des tableaux de bord. Pour les requêtes analytiques les plus fréquemment utilisées, les temps de réponse observés sont restés inférieurs à 800 ms. Le chargement des tableaux de bord interactifs s'est effectué dans un délai compris entre 2 et 3 secondes dans les conditions normales de test.

Une attention particulière a également été accordée aux performances du processus ETL développé avec Talend Open Studio. Les opérations d'extraction, de transformation et de chargement des principales sources de données, notamment *personnel_hist*, *bulletin_h*, *pret_pers* et *dem_cng*, ont été évaluées sur les volumes de données disponibles. Les traitements se sont exécutés dans des délais acceptables et n'ont pas entraîné de dégradation significative des performances du Data Warehouse Oracle.

Enfin, des tests de stabilité ont été effectués afin d'observer le comportement de la solution lors d'une utilisation prolongée. Ces tests ont notamment permis de vérifier la stabilité des différents services déployés à l'aide de Docker et Docker Compose ainsi que le maintien des performances de l'application.

Les résultats obtenus montrent que l'architecture retenue, reposant sur un Data Warehouse Oracle, une API Spring Boot, une interface Angular et un mécanisme de sécurité assuré par Keycloak, présente des performances satisfaisantes pour les besoins de la plateforme

décisionnelle. Toutefois, une augmentation importante du volume de données ou du nombre d'utilisateurs pourrait nécessiter des optimisations supplémentaires, telles que l'amélioration des index, l'utilisation de mécanismes de mise en cache ou l'évolution de l'infrastructure de déploiement.

7.4 Validation des indicateurs décisionnels (KPIs)

La validation des indicateurs clés de performance (KPIs) a constitué une étape importante dans l'évaluation de la plateforme décisionnelle RH & Crédit. Elle avait pour objectif de garantir la fiabilité des résultats obtenus, la pertinence des indicateurs par rapport aux besoins métier ainsi que leur facilité d'interprétation par les utilisateurs finaux. Cette démarche de validation s'est appuyée sur plusieurs vérifications couvrant les aspects métier, techniques et fonctionnels.

Dans un premier temps, les indicateurs ont été définis en fonction des besoins identifiés dans les domaines des RH et du Crédit. Cette étape a permis de sélectionner les KPIs nécessaires au suivi et à l'analyse des principales activités. Parmi les indicateurs retenus figurent notamment l'évolution des effectifs selon le service, le grade et le type administratif ou technique, le coût moyen du personnel, le taux d'absentéisme, le solde des congés ainsi que le volume et l'encours des prêts personnels. La répartition des crédits selon leur type a également été prise en compte afin de fournir une vision synthétique de l'activité de crédit.

Dans un second temps, une validation technique des indicateurs a été réalisée au niveau du Data Warehouse, structuré selon un schéma en étoile. Les résultats des différentes formules de calcul ont été vérifiés à l'aide de requêtes PL/SQL, puis comparés aux données provenant des sources Oracle afin de contrôler la cohérence des agrégations et des valeurs obtenues.

Les processus ETL développés avec Talend Open Studio ont également été vérifiés afin de s'assurer de l'intégrité des données extraites, transformées et chargées dans le Data Warehouse.

Une validation fonctionnelle a ensuite été effectuée à partir de scénarios d'utilisation sur le dashboard Angular. Cette étape avait pour objectif de vérifier que les indicateurs affichés correspondent aux besoins des différents profils utilisateurs, notamment RH, Finance et Direction. Les filtres dynamiques portant sur la période, le service et le grade ont été testés afin de vérifier leur influence sur les résultats affichés. Les différentes visualisations développées avec Chart.js ou ApexCharts ont également été contrôlées afin de s'assurer qu'elles représentent correctement les données et facilitent leur interprétation.

Les écarts éventuellement identifiés au cours de ces vérifications

ont été analysés et corrigés avant la validation finale des indicateurs. Les KPIs retenus ont ensuite été documentés en précisant leur définition, leur mode de calcul, leur source de données et leur objectif d'analyse. Cette formalisation permet d'assurer une interprétation homogène des indicateurs et de faciliter leur utilisation par les différents profils.

Ainsi, la validation des KPIs permet de garantir que les tableaux de bord de la plateforme fournissent des informations fiables, cohérentes et adaptées aux besoins décisionnels. Elle contribue également à renforcer la qualité des analyses et à faciliter le suivi des activités RH et Crédit par les responsables concernés.

7.5 Conclusion

Ce chapitre a permis d'évaluer la fiabilité et la robustesse de la plateforme décisionnelle RH & Crédit à travers une démarche de tests structurée et progressive. Les différents composants de la solution, notamment le Data Warehouse, les processus ETL, l'API REST développée avec Spring Boot et l'application frontend Angular, ont été soumis à plusieurs niveaux de vérification couvrant les tests unitaires, fonctionnels, d'intégration et de performance. Cette démarche a également été complétée par une validation des indicateurs clés de performance (KPIs), afin de vérifier leur cohérence et leur adéquation avec les besoins décisionnels.

Les résultats obtenus montrent que la solution répond aux principales exigences fonctionnelles et non fonctionnelles définies au cours du projet. Les différentes vérifications ont permis de contrôler la cohérence des données, le bon fonctionnement des processus d'intégration, ainsi que les mécanismes d'authentification et de gestion des droits d'accès. Les tableaux de bord développés permettent également une restitution claire et interactive des principaux indicateurs RH et Crédit.

La validation des KPIs pour les différents profils utilisateurs, notamment RH, Finance et Direction, a permis de confirmer leur pertinence pour le suivi et l'analyse des activités de la Banque Extérieure d'Algérie. Les tests réalisés ont ainsi contribué à améliorer la qualité, la stabilité et la fiabilité de la solution.

Ainsi, l'ensemble des vérifications effectuées constitue une étape importante dans l'assurance qualité de la plateforme et permet de disposer d'une solution cohérente, sécurisée et exploitable pour répondre aux besoins d'analyse et d'aide à la décision.

Conclusion générale et perspectives

Conclusion générale

Le présent projet de fin d'études a permis de concevoir et de développer une plateforme décisionnelle dédiée à l'analyse et au pilotage des données relatives aux ressources humaines et au crédit de la Banque Extérieure d'Algérie. Répondant à un besoin métier clairement identifié, la solution proposée repose sur une approche intégrée combinant les principes de la Business Intelligence (BI) et les technologies modernes de développement d'applications web.

La première étape du projet a porté sur la conception et la mise en place d'un Data Warehouse structuré selon un schéma en étoile. Celui-ci est alimenté à partir des sources de données Oracle à travers des processus ETL permettant d'assurer l'extraction, la transformation et le chargement des données. Cette architecture a permis de centraliser les informations relatives aux effectifs, aux charges du personnel, aux absences et aux crédits, tout en fournissant une base structurée et cohérente pour les analyses décisionnelles.

Dans un second temps, une API RESTful a été développée à l'aide de Spring Boot et Spring Data JPA afin d'assurer l'accès aux données analytiques et leur exposition aux différentes fonctionnalités de l'application. L'intégration de Keycloak a permis de renforcer la sécurité de la solution en assurant l'authentification des utilisateurs ainsi que la gestion des droits d'accès selon leurs rôles. Cette couche applicative constitue ainsi une interface intermédiaire entre le Data Warehouse et l'application web, tout en garantissant un accès contrôlé aux informations décisionnelles.

La couche de présentation a ensuite été développée avec Angular et Angular Material, en s'appuyant sur des bibliothèques de visualisation telles que Chart.js et ApexCharts pour représenter graphiquement les indicateurs. L'application fournit aux différents profils utilisateurs, notamment RH, Finance et Direction, des tableaux de bord interactifs permettant de consulter les principaux indicateurs décisionnels et d'affiner les analyses grâce à des filtres dynamiques portant notamment sur la période, le service et le grade. Cette approche facilite l'exploitation des données et améliore la lisibilité des informations mises à disposition des décideurs.

L'ensemble de la solution a fait l'objet d'une démarche de validation couvrant les aspects techniques et fonctionnels. Des tests unitaires, d'intégration, fonctionnels et de performance ont été réalisés afin de vérifier la fiabilité et la robustesse des différents composants.

Parallèlement, les indicateurs décisionnels ont été validés afin de s'assurer de leur cohérence et de leur adéquation avec les besoins des utilisateurs. Les résultats obtenus montrent ainsi que la plateforme répond aux principaux objectifs fixés et constitue un outil permettant de centraliser, analyser et restituer efficacement les informations nécessaires au pilotage des activités RH et Crédit.

Perspectives

Bien que la solution développée réponde aux objectifs du projet, plusieurs perspectives d'évolution peuvent être envisagées.

Une première amélioration consisterait à intégrer un reporting avancé (JasperReports) permettant la génération automatique de rapports PDF paramétrables. L'ajout de techniques d'intelligence artificielle et de Machine Learning pourrait également permettre de prévoir l'absentéisme, de détecter des anomalies dans les crédits ou de proposer des recommandations.

Sur le plan technique, un déploiement sous Kubernetes améliorerait la scalabilité et la disponibilité de la plateforme. Le périmètre fonctionnel pourrait être étendu à d'autres domaines (gestion des risques, conformité, performance commerciale). Enfin, la mise en place d'alertes proactives, déclenchées lorsque certains indicateurs dépassent des seuils définis, renforcerait l'aide à la décision.

En définitive, ce projet a permis de répondre à un besoin concret de la Banque Extérieure d'Algérie en mobilisant des compétences en Business Intelligence, ingénierie des données, développement et sécurité. La solution réalisée constitue une base évolutive pouvant être enrichie pour accompagner durablement les besoins décisionnels de l'organisation.

Bibliographie

Références citées dans le rapport

ANGULAR. Angular Documentation — Routing and Single Page Applications. Documentation officielle. Disponible sur :

<https://angular.dev/>

FIELDING, Roy Thomas. Architectural Styles and the Design of Network-based Software Architectures. Doctoral dissertation, University of

California, Irvine, 2000.

HARDT, Dick. The OAuth 2.0 Authorization Framework. RFC 6749, Internet Engineering Task Force, 2012.

INMON, William H. Building the Data Warehouse. 4th edition, Wiley, 2005.

KEYCLOAK. Keycloak Server Administration Guide. Documentation officielle. Disponible sur : <https://www.keycloak.org/documentation>

KIMBALL, Ralph; ROSS, Margy. The Data Warehouse Toolkit: The Definitive Guide to Dimensional Modeling. 3rd edition, Wiley, 2013.

MICROSOFT. Dimensional Modeling — Star Schema. Microsoft Learn. Documentation officielle.

ORACLE. What Is a Data Warehouse? Oracle, 2023. Documentation officielle.

SAKIMURA, Nat; BRADLEY, John; AGARWAL, Naveen. Proof Key for Code Exchange by OAuth Public Clients. RFC 7636, IETF, 2015.

SAKIMURA, Nat; BRADLEY, John; JONES, Michael; DE MEDEIROS, Breno; MORTIMORE, Chuck. OpenID Connect Core 1.0

incorporating errata set 2. OpenID Foundation, 2023.

SPRING SECURITY. OAuth 2.0 Resource Server — Spring Security Reference. Documentation officielle. Disponible sur :

<https://docs.spring.io/spring-security/reference>



ECOLE SUPÉRIEURE PRIVÉE D'INGÉNIERIE ET DE TECHNOLOGIES

www.esprit.tn - E-mail : contact@esprit.tn

Siège Social : 18 rue de l'Usine - Charguia II - 2035 - Tél. : +216 71 941 541 - Fax. : +216 71 941 889

Annexe : Z.I. Chotrana II - B.P. 160 - 2083 - Pôle Technologique - El Ghazala - Tél. : +216 70 685 685 - Fax. : +216 70 685 454